Module 1: Introduction to R

Lesson 1: R and RStudio (slide 2)

Lesson 2: Basic concepts in R (slide 24)

Lesson 3: Data manipulation with dplyr (slide 105)

Lesson 4: Visualization with ggplot2 (slide 123)

Lesson 1: R and RStudio

Set up the tools, then explore the workspace.

R and RStudio

R is the language and computing environment.
RStudio is an interface for writing and running R.
Use them together for analysis and graphics.

History of R

Ross Ihaka and Robert Gentleman developed R in Auckland.
R draws on S and lexical scoping from Scheme.
The name refers to its creators and to S.
1993: public announcement. 1995: source released under the GPL.

Analysis, graphics, and packages

Statistical methods: models, tests, time series, and more.
Graphics help explore and communicate data.
Packages extend R with functions and datasets.
CRAN distributes contributed packages.

Programming, data, and support

Objects hold vectors, matrices, arrays, lists, and tables.
Conditions, loops, and functions organize calculations.
Input and output connect R to files and other systems.
Documentation, forums, and contributors support users.

Advantages of R

Free, open source, and available on Windows, macOS, and Linux.
An active community maintains tools and documentation.
R can connect with languages such as C, C++, and Python.

R in a data analysis workflow

Prepare data with tools such as dplyr and tidyr.
Apply statistical methods to answer a question.
Use ggplot2 to communicate patterns and results.

The next lessons follow this workflow.

Careers using R: analysis and business

Data analyst: prepare and interpret data.
Data scientist: develop models and predictions.
Quantitative analyst: study financial data and risk.
Statistician: design studies and analyze evidence.
Market research analyst: study customers and markets.

Careers using R: research and modeling

Bioinformatician / biostatistician: analyze biological data.
Machine learning engineer: develop learning algorithms.
Econometrician: model economic relationships.
Environmental scientist: analyze environmental data.
Public health analyst: assess population health evidence.

Careers using R: applied decisions

Educational researcher: evaluate learning and programs.
Actuary: quantify insurance and financial risk.
Operations research analyst: improve decisions and processes.
Sports analyst: examine performance and strategy.
Clinical researcher: analyze studies and trials.

Installation order

1. Download and install R from CRAN.
2. Install RStudio Desktop from Posit.
3. Open RStudio and check that R starts.

Choose installers compatible with your operating system.

Installing R on Windows

Open CRAN and choose the Windows base installer.
Run the .exe and follow the setup prompts.
Choose the installation folder and components.
Finish setup, then install RStudio Desktop.

Installing R on macOS

Choose the R .pkg matching your Mac and macOS.
Run the installer and follow its prompts.
Use the current CRAN instructions for compatibility.
Then install RStudio Desktop.

The Quarto screenshots and menu wording may reflect an older release.

Installing R on Linux

Installation depends on the Linux distribution.
For Debian / Ubuntu, use the package manager.
An optional CRAN repository must match your distribution.

Terminal commands: sudo apt update, then sudo apt install r-base

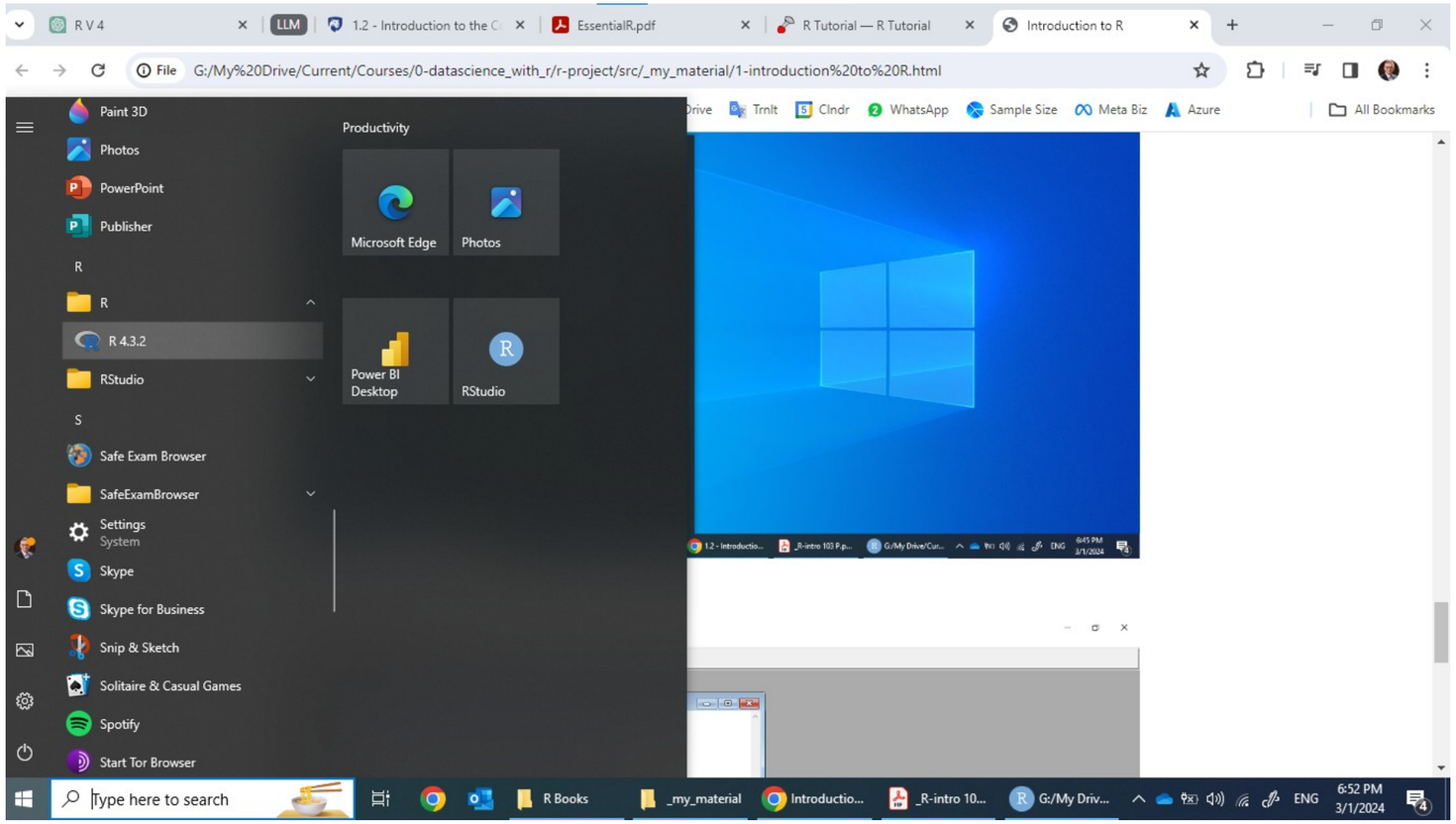
Installation check

Open R or the RStudio Console.
Inspect the R version, platform, and attached packages.

```
# sessionInfo() reports the current R session.
# Use its version and platform details when diagnosing setup issues.
sessionInfo()
```

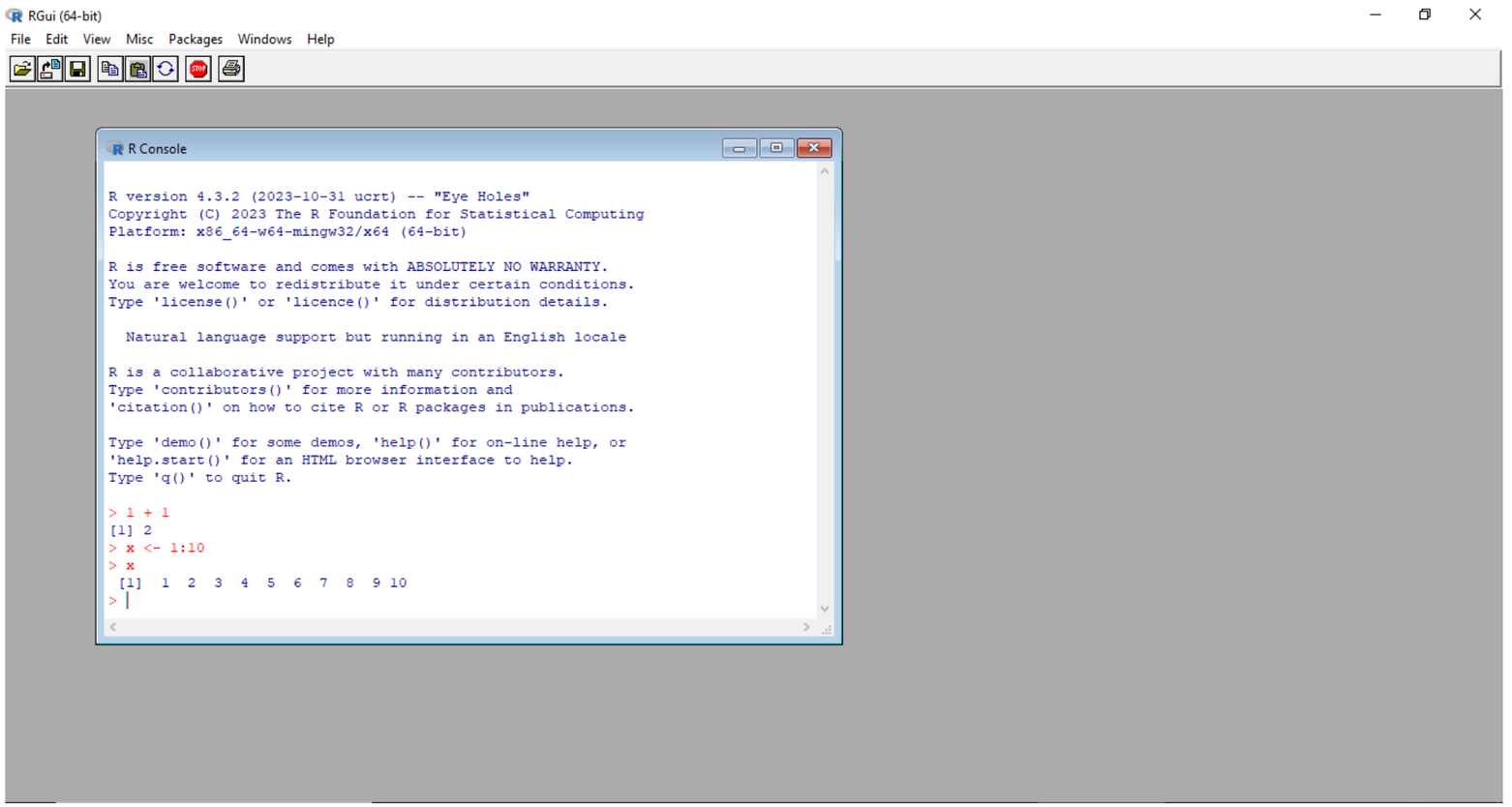
Opening the R Console

On Windows, find R in the Start menu.
Open the application to reach the command prompt.



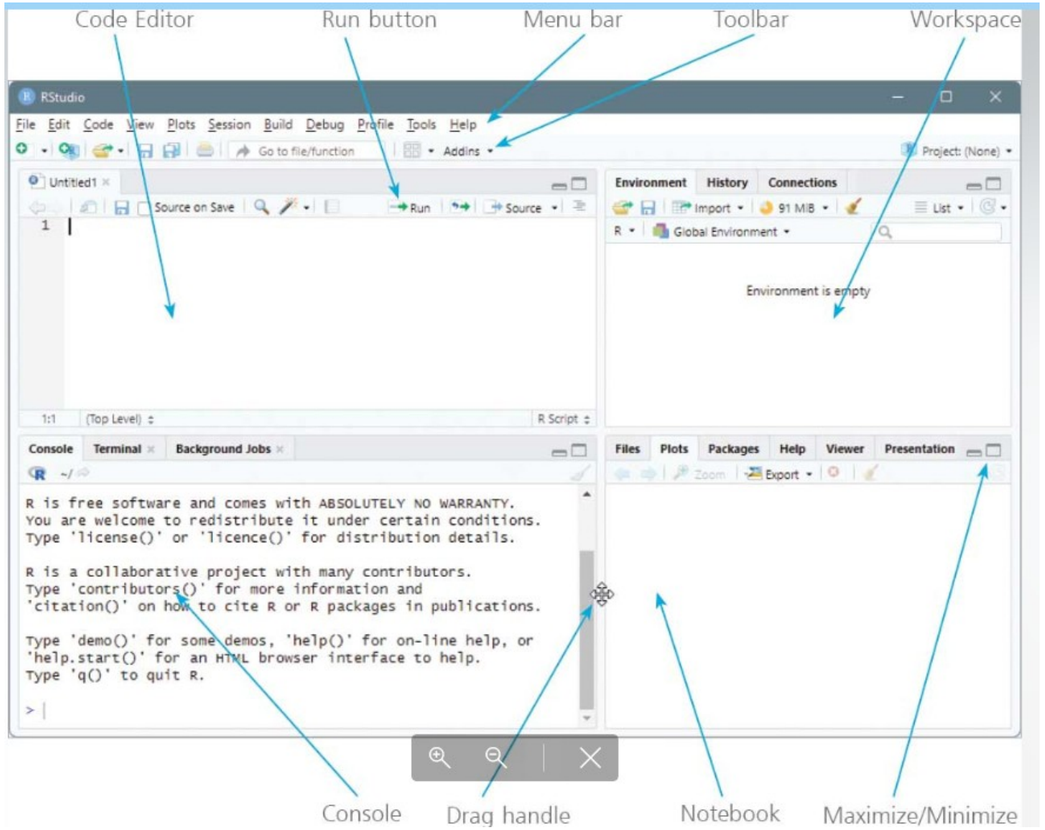
The R Console

Type commands after the > prompt.
R evaluates them and prints results.



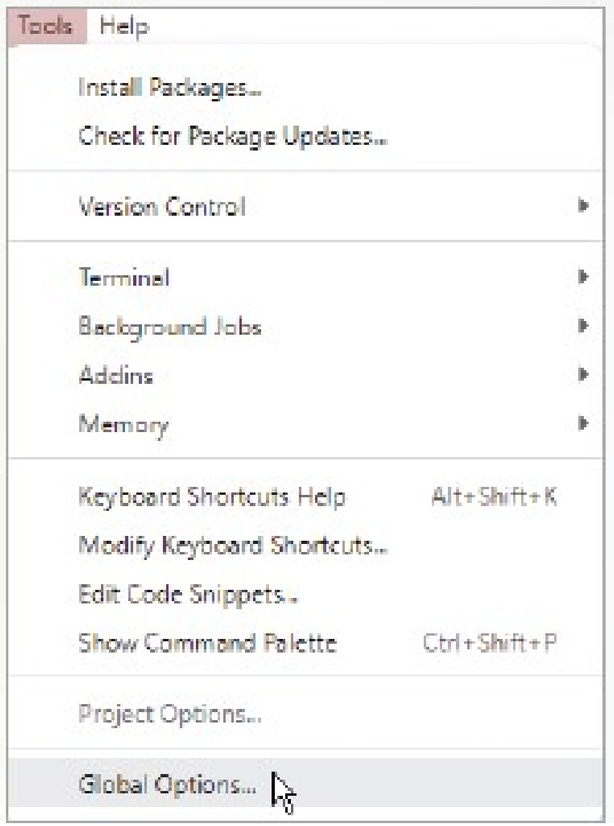
The RStudio workspace

Find the Source editor, Console, and Environment.
Use Files, Plots, Packages, and Help while working.



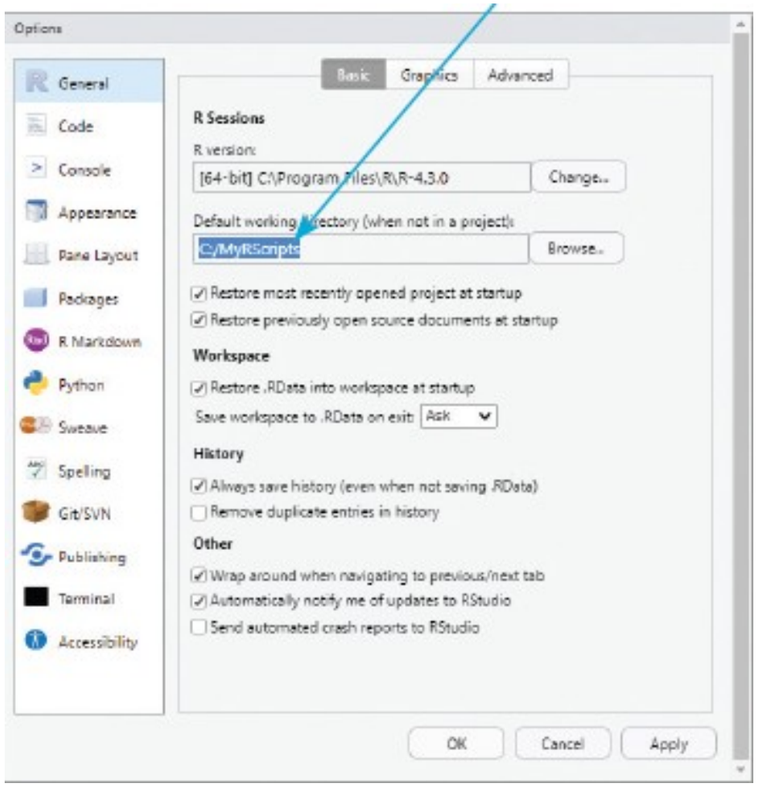
Working directory settings

The working directory is the starting point for relative paths.
In RStudio, find the General options.



Default working directory

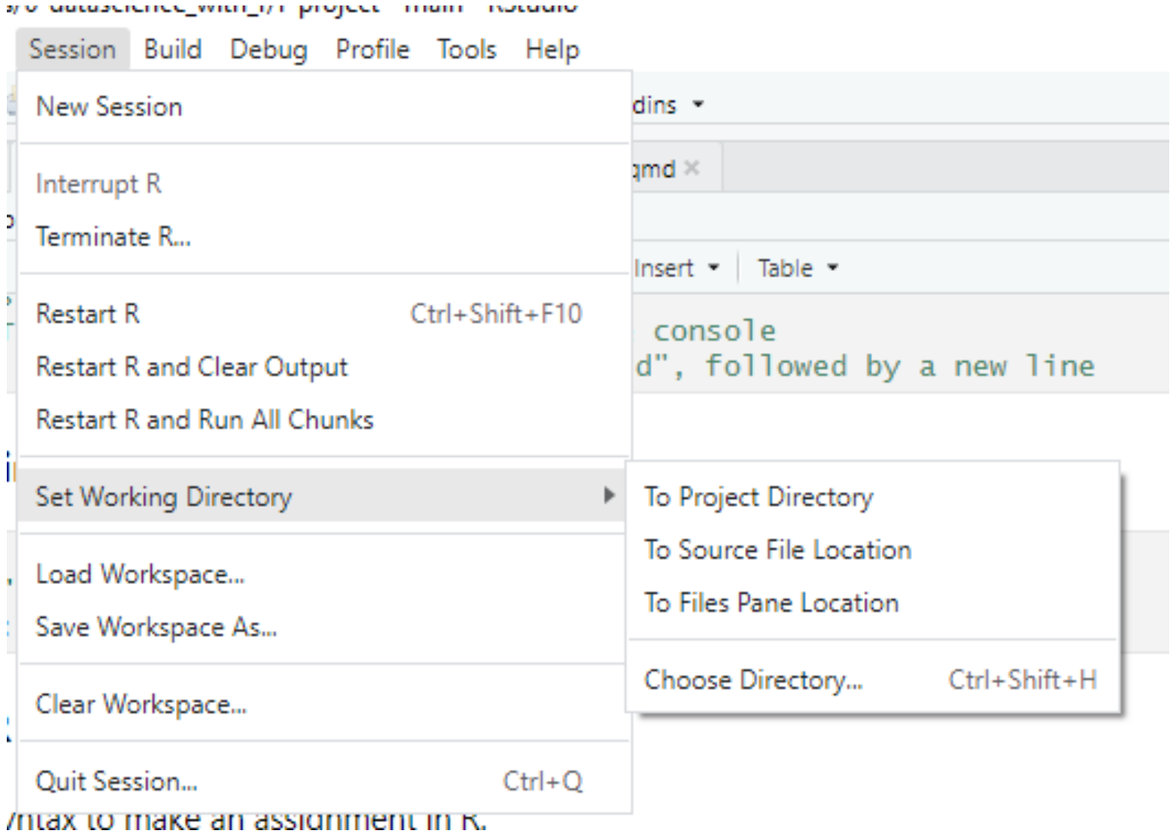
The lesson illustrates choosing a default folder.
For this course, open the supplied RStudio project.



Course CSV paths resolve from the 20261 project folder.

Changing the working directory

The Session menu also provides directory controls.
Check the active folder when a file cannot be found.



Lesson 1 references

Penn State STAT 484
Penn State Essential R notes
Cyclismo R tutorial
R for Data Analysis in easy steps / R Programming Essentials

Links and the original attribution are in the Quarto lesson.

Lesson 2: Basic concepts in R

With the workspace ready, start writing and reading R code.

Hello, World

Run the same short program in the Console and in a script.
An assignment stores a value. print() displays it.

```
# <- stores the text in the object myString.
# print() displays that stored object in the Console.
myString <- "Hello, World!"
print(myString)
```

The command prompt and printed output

In a system terminal, R starts an interactive R session.
Inside R, the > prompt waits for a command.
The printed [1] marks the first value shown on that output line.

System-terminal commands and R expressions use different prompts.

Reusable R scripts

Save the Hello World commands in test.R.
Run the script in RStudio or use Rscript test.R in a terminal.
A script keeps a repeatable record of the commands.

Comments explain the code

Text following # is a comment.
R ignores comments when it evaluates the program.

```
# Comments describe intent and important assumptions.
# Keep executable expressions on their own lines when possible.
# My first program in R Programming
```

Variables and objects

A name refers to an R object.
Assignment creates or replaces the value under that name.
R determines the type from the assigned object.
Types describe values; structures organize those values.

Next: numeric, complex, character, and logical values.

Numeric and integer values

Numeric values commonly use double precision.
The L suffix explicitly creates an integer.

```
# 42L is stored as an integer; 72.5 is stored as a double.
# <- assigns each value to the name on its left.
count <- 42L
weight <- 72.5
```

Complex values

A complex value has real and imaginary components.

```
# The suffix i represents the imaginary component.
# This value has real part 3 and imaginary part 4.
complex_num <- 3 + 4i
```

Character values

Character values store text in quotation marks.
A single character string can contain many letters.

```
# Quotation marks distinguish text from an object name.
# Both assignments create character values.
name <- "Alice"
sentence <- "Hello, World!"
```

Logical values: Boolean results

TRUE and FALSE represent logical values.
Comparisons produce logical results.
Use full names rather than the reassignable T and F aliases.

```
# > tests whether the left value exceeds the right value.
# | means OR; & requires both elementwise conditions to be TRUE.
is_valid <- TRUE
test_result <- 5 > 3
a <- TRUE
b <- FALSE
a | b
a & b
```

Case sensitivity

age, Age, and AGE are different names.
Function names also require the correct capitalization.

```
# print() displays a value; capitalized Print() is a different name.
# Only age is defined here, so the last two calls intentionally fail.
age <- 25
print(age)
# Intentional error examples:
print(Age)
Print(age)
```

Run error demonstrations separately from a working analysis.

A table with different column types

The lesson constructs a small BMI example table.
Rows align the values belonging to each person.

```
# c() combines values; data.frame() aligns the columns into a table.
# Columns can have different types but must have compatible lengths.
BMI <- data.frame(
  gender = c("Male", "Male", "Female"),
  height = c(152, 171.5, 165),
  weight = c(81, 93, 78),
  Age = c(42, 38, 26)
)
```

Illustrative records from the lesson; source does not state height/weight units.

Inspecting type and structure

class() reports the object class.
typeof() reports the underlying storage type.
str() shows the structure and example values.

```
# class() describes how an object behaves; typeof() describes storage.
# str() displays columns, their types, and a compact preview.
class(BMI)
typeof(BMI)
str(BMI)
```

Comparison operators

< and >: less than and greater than
<= and >=: less than or equal and greater than or equal
==: equal to
!=: different from

A comparison tests a relationship and returns a logical result.

Combining logical conditions

& requires both conditions.
| accepts either condition.
! reverses TRUE and FALSE.

These elementwise operators also work on logical vectors.

Organizing values

Vectors organize values along one dimension.
Matrices and arrays organize values in more dimensions.
Lists can combine unlike objects.
Data frames organize observations and variables.
Factors represent categories.

Start with vectors, then move to the other structures.

Vectors

An atomic vector holds values of one common type.
Combining incompatible types can coerce them to a common type.

```
# c() combines values in their supplied order.
# These examples create numeric, character, and logical vectors.
ages <- c(25, 30, 45)
colors <- c("red", "green", "blue")
answers <- c(TRUE, FALSE, TRUE)
```

The lesson data table

Patient ID identifies the observation.
Height (cm), weight (kg), and blood pressure (mmHg) are columns.
Each column can be represented as a vector.

Patient ID	Height (cm)	Weight (kg)	Systolic (mmHg)	Diastolic (mmHg)
10200	165	71	120	80
3600	180	92	110	85
8437	172	76	118	82
5911	167	82	115	78

Illustrative table reproduced from the lesson; no clinical interpretation.

Creating sequences

Sequences are ordered values following a pattern.
Use : for steps of one.
Use seq() to control spacing or length.
Use rep() to repeat values.

The following examples distinguish spacing from repetition.

Consecutive values with :

The colon operator steps by one toward the endpoint.

```
# 1:10 creates the integers from 1 through 10.
# print() displays the sequence saved by the assignment.
seq_1_to_10 <- 1:10
print(seq_1_to_10)
```

Sequence spacing with seq()

from and to set the endpoints.
by specifies the step size.

```
# seq() generates values with the requested spacing.
# by = 2 gives 1, 3, 5, 7, 9; the endpoint need not be reached.
seq_by_2 <- seq(from = 1, to = 10, by = 2)
print(seq_by_2)
```

Sequence length with seq()

length.out requests the number of evenly spaced values.

```
# length.out = 3 requests three values including the endpoints.
# print() displays 5, 7.5, and 10.
seq_length_out <- seq(from = 5, to = 10, length.out = 3)
print(seq_length_out)
```

Repeating a whole vector

times repeats the complete input vector.

```
# rep() repeats supplied values; 1:3 creates 1, 2, 3.
# times = 3 repeats that whole sequence three times.
rep_times <- rep(x = 1:3, times = 3)
print(rep_times)
```

Repeating individual elements

each repeats every element before moving to the next.

```
# rep() repeats values; each = 3 repeats each input three times.
# Compare 1,1,1,2,2,2,3,3,3 with the times example.
rep_each <- rep(x = 1:3, each = 3)
print(rep_each)
```

Combining sequences

Use c() to join generated sequences.

```
# seq() creates steps of one here; c() joins the two sequences.
# print() shows 1 to 5 followed by 8 to 10, preserving that order.
combined_seq <- c(seq(1, 5), seq(8, 10))
print(combined_seq)
```

Vector practice: combinations

Combine 1, 2, 3 with "a", "b", "c".
Take every second number from 1 to 10.
Repeat 1,2,3 twice, followed by 4,5,6 twice.
Start with 1,2,3, then repeat 4 five times.

Pause here and complete the matching Quarto prompts.

Vector practice: patterns

Create the first 10 even numbers.
Create the first 10 odd numbers.
Use seq() to generate 1 to 10, repeating each value twice.
Use rep() to repeat the sequence 1,2,3 three times.

Keep these as student attempts; return to the lesson for practice.

Vector indexing

Square brackets select values.

R counts positions from 1.

Start with the same vector throughout these examples.

```
# c() creates five values in the displayed order.
# Positions 1 through 5 refer to those five values.
numbers <- c(10, 20, 30, 40, 50)
```

One vector element

Select the third position.

```
# [3] extracts the value in position 3.
# print() displays 30 from the vector defined on the previous slide.
third_number <- numbers[3]
print(third_number)
```

Several vector positions

Supply a vector of positions inside brackets.

```
# c(1, 3, 5) identifies three selected positions.
# The bracket expression returns 10, 30, and 50.
selected_numbers <- numbers[c(1, 3, 5)]
print(selected_numbers)
```

A consecutive range

Use a sequence of indices to select a range.

```
# 2:4 identifies positions 2, 3, and 4.
# The bracket expression returns 20, 30, and 40.
range_numbers <- numbers[2:4]
print(range_numbers)
```

Values matching a condition

A logical condition can select values directly.

```
# numbers > 25 marks qualifying values as TRUE.
# Logical indexing keeps 30, 40, and 50 in their original order.
filtered_numbers <- numbers[numbers > 25]
print(filtered_numbers)
```

Positions matching a condition

which() returns positions rather than the selected values.

```
# which() returns the integer positions of TRUE conditions.
# The result is 3, 4, 5; compare it with values 30, 40, 50.
which(numbers > 25)
```

Excluding vector positions

Negative indices omit the specified positions.

```
# A negative index excludes positions rather than selecting them.
# Parentheses make -(2:4) exclude positions 2 through 4 together.
exclude_second <- numbers[-2]
exclude_range <- numbers[-(2:4)]
print(exclude_second)
print(exclude_range)
```

Named vector elements

Names provide another way to select vector values.

```
# c() creates a named vector; quoted names select its elements.
# Single brackets preserve the selected names in the result.
fruits <- c(apple = 10, banana = 5, cherry = 8)
banana_count <- fruits["banana"]
some_fruits <- fruits[c("apple", "cherry")]
print(banana_count)
print(some_fruits)
```

Arithmetic on every element

An arithmetic operation applies to each vector entry.
Reassigning a saves the updated values.

```
# c() creates the vector; each arithmetic step changes every value.
# b <- a saves the current values under another name.
a <- c(1, 2, 3, 4)
a <- a + 5
a <- a - 10
a <- a / 5
b <- a
```

Powers, roots, exponentials, and logarithms

Functions can also operate on an entire numeric vector.

```
# ^ raises powers; sqrt() takes roots; exp() computes e to a power.
# log() uses natural logarithms by default; check input domains.
a <- c(1, 2, 3, 4)
a ^ 2
sqrt(a)
exp(a)
log(a)
exp(log(a))
```

Combining vector expressions

Parentheses determine the calculation order.

Vector pairs combine element by element.

```
# sqrt() and exp() act on their inputs before the surrounding arithmetic.
# Check zero denominators and square-root domains before interpreting results.
c <- (a + sqrt(a)) / (exp(2) + 1)
a + b
a * b
a / b
(a + 3) / (sqrt(1 - b) * 2 - 1)
```

These continue the lesson objects; they illustrate syntax, not a data finding.

Unequal lengths and recycling

R recycles the shorter vector.

A nonmultiple length produces a warning.

Equal lengths are the clearest choice for paired observations.

```
# c() creates unequal vector lengths of three and four.
# This intentional warning example recycles a and warns about the length mismatch.
a <- c(1, 2, 3)
b <- c(10, 11, 12, 13)
a + b
```

Numerical summaries

mean() and median(): center
quantile(): percentiles
var() and sd(): spread
min() and max(): endpoints

```
# quantile() uses type = 7 by default here; var() divides by n - 1.
# sd() is the sample standard deviation, in the original measurement units.
a <- 1:10
mean(a); median(a); quantile(a)
var(a); sd(a)
min(a); max(a)
```

Module 2 develops the interpretation of these measures.

An overview with summary()

Numerical summaries include quartiles, center, and endpoints.
For a data frame, summary() works column by column.

```
# summary() chooses an overview suited to the object type.
# For numeric columns it reports min, quartiles, median, mean, and max.
summary(a)
summary(mtcars)
```

Sorting vector values

sort() returns values in increasing order by default.

```
# c() combines the unsorted input values.
# sort() orders values; decreasing = TRUE reverses the default order.
a <- c(2, 4, 6, 3, 1, 5)
b <- sort(a)
c <- sort(a, decreasing = TRUE)
```

Vectorization and an equivalent loop

Both approaches multiply matching elements.
Vectorization expresses the operation directly.

```
# rnorm() generates artificial normal draws; rep() allocates missing entries.
# The for loop fills each position; a * b does the operation at once.
N <- 100
a <- rnorm(N)
b <- rnorm(N)
c <- a * b
d <- rep(NA, N)
for (i in 1:N) {
  d[i] <- a[i] * b[i]
}
```

Comparing execution times

Use 100,000 values, as in the lesson.
Compare measured execution time on your own computer.
Timings vary between computers and runs.

```
# rnorm() generates artificial inputs; rep() allocates the loop result.
# system.time() measures CPU and elapsed time; print() displays those timings.
N <- 100000
a <- rnorm(N); b <- rnorm(N)
vectorized_time <- system.time({ c <- a * b })
de_vectorized_time <- system.time({
  d <- rep(NA, N)
  for (i in 1:N) d[i] <- a[i] * b[i]
})
print(vectorized_time)
print(de_vectorized_time)
```

Beyond a vector

The same values can be organized in different structures.
Choose a structure that fits the question and the data.

Next: matrices, arrays, lists, and data frames.

Matrices

A matrix has rows and columns and one underlying type.
R fills columns first by default.

```
# matrix() arranges six sequential values into a rectangular object.
# nrow = 3 and ncol = 2 specify three rows and two columns.
matrix_data <- matrix(1:6, nrow = 3, ncol = 2)
```

The source also mentions an NBA example but supplies no linked file.

Arrays

An array can have more than two dimensions.

```
# array() organizes values into the dimensions supplied by c().
# dim = c(3, 4, 2) gives three rows, four columns, and two layers.
array_data <- array(1:24, dim = c(3, 4, 2))
```

Lists

A list can hold objects of different types and lengths.

```
# list() keeps unlike objects together; c() combines the score values.
# [1] returns a sublist; [[1]] extracts its first element; print() displays it.
list_data <- list(
  name = "Alice", age = 25, scores = c(90, 80, 85))
print(list_data[1])
```

Data frames

A data frame is a rectangular collection of columns.
Columns can have different types. Rows align observations.

```
# c() supplies the column values; data.frame() combines them into a table.
# Keep one observation per row; the source does not state Weight units.
df <- data.frame(
  Name = c("Alice", "Bob"),
  Age = c(25, 30),
  Weight = c(55.5, 85.7)
)
```

Built-in practice datasets

Use data() to browse datasets and ?mtcars for help.
View() opens a table viewer in RStudio.
mtcars is the lesson example for table access.

```
# data() lists available datasets; ? opens the named help page.
# View() opens an interactive viewer and is a manual RStudio step.
data()
?mtcars
View(trees)
```

mpg means miles per US gallon; wt is vehicle weight in 1,000 lb.

Column access: vector or table

mtcars$mpg extracts a vector.
mtcars["mpg"] keeps a one-column data frame.

```
# $ extracts a column vector; ["mpg"] retains a data frame.
# print() shows each result; the legacy name is misleading: mpg is fuel economy.
miles_per_hour <- mtcars$mpg
print(miles_per_hour)
miles_per_hour <- mtcars["mpg"]
print(miles_per_hour)
```

The original object name is retained here, but mpg is not miles per hour.

Row and column indexing

Use table[rows, columns].

A blank dimension keeps all its entries.

```
# c() selects several columns; : creates consecutive index ranges.
# With a comma, the first position selects rows and the second selects columns.
subset_df <- mtcars[c("mpg", "cyl")]
first_row <- mtcars[1, ]
mpg <- mtcars[2, "mpg"]
mtcars[5:7, ]
mtcars[20:30, 8:10]
```

Indexing variations

One index on a data frame selects columns.
A comma separates row selection from column selection.

```
# : generates consecutive positions; c() combines individual positions.
# Compare each one-index form with the corresponding row/column form.
mtcars[2:9]
mtcars[, 2:9]
mtcars[2, 9]
mtcars[2:9, ]
mtcars[c(2, 6)]
mtcars[, c(2, 6)]
mtcars[c(2, 6), ]
mtcars[2:6]
mtcars[5, ]
```

Row names and identifiers

Row names can identify a record.

They can also be copied into an explicit column.

```
# rownames() retrieves record labels; [1] selects the first label.
# head() previews six rows; these row names identify models, not just brands.
car_names <- rownames(mtcars)
first_car_name <- rownames(mtcars)[1]
datsun_710 <- mtcars["Datsun 710", ]
mtcars$car_brand <- rownames(mtcars)
head(mtcars)
mtcars["Fiat 128", ]
```

Conditions and table dimensions

A logical row condition selects observations.
For data frames, length() counts columns.
nrow() and ncol() explicitly count rows and columns.

```
# == creates logical conditions; am = 0 denotes automatic transmission.
# length() counts vector elements or table columns; nrow()/ncol() count dimensions.
m <- mtcars[mtcars$cyl == 4, ]
length(m)
length(mtcars[, 1])
nrow(mtcars)
ncol(mtcars)
nrow(mtcars[mtcars$am == 0, ])
```

Filtering against a calculated value

A condition can compare values with a computed summary.

```
# c() combines values; mean() calculates their average.
# subset() selects matching rows using column names; print() displays them.
mean(c(40, 60, 40, 60))
mtcars[mtcars$mpg < mean(mtcars$mpg), ]
more_than_4_cyl <- subset(mtcars, cyl > 4)
print(more_than_4_cyl)
```

Student table: construction

The lesson combines aligned name, age, and gender vectors.

```
# c() builds the three vectors; data.frame() aligns them by row.
# print() displays the illustrative table used in the next access examples.
name <- c("John", "Alice", "Bob", "Eva")
age <- c(23, 25, 30, 22)
gender <- c("Male", "Female", "Male", "Female")
students <- data.frame(name, age, gender)
print(students)
```

Student table: access and filter

Compare named columns, positions, and a logical row filter.

```
# $ extracts a vector; ["gender"] retains a one-column table.
# [row, column] chooses positions; > 23 marks rows retained by the filter.
print(students$age)
print(students["gender"])
print(students[2, 1])
print(students[3, ])
print(students[, 2])
older_students <- students[students$age > 23, ]
print(older_students)
```

Output, input, and the workspace

After storing and accessing values, consider how to interact with R. Display results, request input, and manage saved objects.

Printing and joining text

print() displays an R object.
cat() writes values together as output.
paste() returns joined character strings.

```
# paste() uses sep between inputs; print() displays the resulting value.
# cat() writes text; "\n" explicitly adds a new output line.
print("Hello, World!")
cat("Hello,", "World!", "\n")
result <- paste("Hello", "World", sep = " ")
print(result)
cat(result, "\n")
```

Interactive input

Try this in the Console rather than an unattended Quarto render.

```
# readline() waits for an interactive line and returns character text.
# cat() writes the response with the supplied label.
name <- readline("Please enter your name: ")
cat("Your name is:", name)
```

Assignment syntax

<- is the preferred course style.

-> assigns toward the right. = can also assign.

== compares values rather than assigning.

```
# The receiving side of an assignment must be an assignable object.
# The final line intentionally fails because literal 1 cannot receive a value.
x <- 1
1 -> x
x = 1
# Intentional invalid assignment:
1 = x
```

Inspecting and clearing the workspace

ls() lists object names. rm() removes specified objects.
The RStudio broom or workspace menu also clears objects.
Restart R when you need a fresh session.

```
# ls() returns names; rm(x, y) removes only those named objects.
# rm(list = ls()) clears the current environment deliberately.
x <- 1
y <- 2
ls()
rm(x, y)
ls()
rm(list = ls())
```

Restart shortcut: Ctrl+Shift+F10; use Cmd+Shift+F10 on macOS.

Flow of control

So far, code has applied operations to objects.
Loops repeat steps. Conditions choose steps.
Functions package steps for reuse.

Begin with repetition, then branching, then functions.

The for loop

A for loop visits each value in a sequence.

```
# 1:5 supplies five values; for assigns each in turn to num.
# ^2 squares the current value; print() displays each square.
numbers <- 1:5
for (num in numbers) {
  print(num^2)
}
```

Loop indexing practice

Replace the stars so the loop visits every position of x.
Keep the exercise unfinished until students attempt it.

```
# This is a practice template, not executable R until the stars are replaced.
# x[i] selects a position; print() displays the value in that position.
x <- 50:60
for (i in *:*) {
  print(x[i])
}
```

The while loop

Repeat while a condition remains TRUE.
Update the value used in the condition so the loop can stop.

```
# while tests the condition before each repetition.
# paste() joins the label and value; print() displays it; subtraction updates it.
countdown <- 5
while (countdown > 0) {
  print(paste("Counting down:", countdown))
  countdown <- countdown - 1
}
print("Lift off!")
```

The repeat loop

repeat keeps running until break exits.

The example searches above 100 for a multiple of 13.

```
# %% calculates a remainder; == 0 tests exact divisibility.
# if chooses the stopping branch; paste()/print() show it; break exits the loop.
number <- 101
repeat {
  if (number %% 13 == 0) {
    print(paste("Found a number divisible by 13:", number))
    break
  }
  number <- number + 1
}
```

Nested loops

An outer loop can visit rows while an inner loop visits columns.

```
# matrix() fills columns first; nrow()/ncol() supply the index bounds.
# Each nested-loop pass doubles one cell; print() displays the final matrix.
my_matrix <- matrix(1:9, nrow = 3)
for (row in 1:nrow(my_matrix)) {
  for (col in 1:ncol(my_matrix)) {
    my_matrix[row, col] <- my_matrix[row, col] * 2
  }
}
print(my_matrix)
```

Conditional branching

Conditions choose which code to run.
An if condition must resolve to one nonmissing logical value.
Use else and else if to handle alternative cases.

A single if condition

Run the body when the condition is TRUE.

```
# if checks whether number is greater than zero.
# print() runs only when that condition is TRUE.
number <- 5
if (number > 0) {
  print("The number is positive.")
}
```

An alternative with else

else handles the remaining case.

```
# if selects the first branch only for a positive number.
# else selects the alternative; print() displays its message.
number <- -3
if (number > 0) {
  print("The number is positive.")
} else {
  print("The number is not positive.")
}
```

Several conditions with else if

Test conditions in sequence.

The first TRUE branch runs.

```
# else if tests a second condition only when the first is FALSE.
# The final else covers zero here; print() displays the selected message.
number <- 0
if (number > 0) {
  print("The number is positive.")
} else if (number < 0) {
  print("The number is negative.")
} else {
  print("The number is zero.")
}
```

Matching labels with switch()

Choose a branch by matching a character value.

```
# switch() selects the branch matching day; print() displays its text.
# The last unnamed branch is a fallback for an unmatched character label.
day <- "Tue"
switch(day,
  "Mon" = {print("Start of the work week")},
  "Tue" = {print("Second day of the work week")},
  "Wed" = {print("Middle of the work week")},
  "Thu" = {print("Approaching the end")},
  "Fri" = {print("Last day of the work week")},
  {print("It must be the weekend")}
)
```

These are the lesson labels, not a calendar convention for every country.

Creating reusable functions

A function bundles a calculation under a name.
Arguments supply inputs. The function body performs the work.
The result can be reused by later code.

Function inputs, body, and result

The example adds two supplied numbers.

```
# function(a, b) defines two inputs; return() chooses the output.
# add_two_numbers() calls that calculation; print() displays its result.
add_two_numbers <- function(a, b) {
  result <- a + b
  return(result)
}
print(add_two_numbers(5, 3))
```

Arguments and return values

A function returns its last evaluated expression by default.
return() explicitly selects a result or returns early.

```
# function() defines the input; paste() joins text using a space by default.
# return() passes the greeting back; print() shows the call result.
greet_person <- function(name) {
  greeting <- paste("Hello,", name)
  return(greeting)
}
print(greet_person("Alice"))
```

Default arguments

A default value makes an argument optional.

```
# function() sets exponent = 2 unless the caller supplies another value.
# ^ computes the power; return() passes it back; print() shows 16 and 64.
raise_to_power <- function(base, exponent = 2) {
  result <- base ^ exponent
  return(result)
}
print(raise_to_power(4))
print(raise_to_power(4, 3))
```

Returning several results

Return a named list to keep related results together.

```
# mean() averages and sum() totals; list() combines the named results.
# function()/return() define the calculation; c() supplies inputs; print() shows it.
calculate_stats <- function(numbers) {
  mean_val <- mean(numbers)
  sum_val <- sum(numbers)
  return(list(mean = mean_val, sum = sum_val))
}
stats <- calculate_stats(c(1, 2, 3, 4, 5))
print(stats)
```

Anonymous functions

A short function can be supplied directly to another function.

```
# function(x) x * 2 defines a calculation without a separate function name.
# sapply() applies it to each value and simplifies compatible results; print() shows them.
numbers <- 1:5
doubled_numbers <- sapply(numbers, function(x) x * 2)
print(doubled_numbers)
```

Local scope

An object created inside a function is local to that call.
Return the result when you want to use it outside.

```
# function() creates local x and y; return() exposes the computed result.
# increment() performs the call; print() shows it; y is not created outside.
increment <- function(x) {
  y <- x + 1
  return(y)
}
incremented_value <- increment(5)
print(incremented_value)
```

Continue with the basic-concepts exercises in Quarto.

Lesson 3: Data manipulation with dplyr

Use the R foundations to prepare and summarize the textbook table.

Restaurant case

Select restaurants by quality and price, then summarize prices by quality.
One row is one restaurant, identified by restaurant_id.
Prices are in USD. Ratings: Good, Very Good, Excellent.

نختار المطاعم حسب الجودة والسعر، ثم نلخّص الأسعار حسب الجودة.
يمثّل كل صف مطعمًا واحدًا يحدّده restaurant_id.
الأسعار بالدولار الأمريكي. التقييمات: جيد، وجيد جدًا، وممتاز.

Historical textbook data; statistical interpretation continues in Module 2.

Load the checked CSV

Load dplyr and read the teaching CSV.
Work from the 20261 RStudio project.

```
# library() loads dplyr; read.csv() reads a CSV with names in its first row.
# head() previews six rows; glimpse() shows dimensions, types, and sample values.
library(dplyr)
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
head(restaurants)
glimpse(restaurants)
```

Keep the source CSV unchanged; save derived tables under new names.

Recreate the ordered rating

A CSV stores labels but cannot preserve R factor metadata. Explicitly record the category order after import.

```
# factor() records categories; c() supplies their permitted order.
# ordered = TRUE marks ordinal levels; $ chooses the rating column.
restaurants$quality_rating <- factor(
  restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"),
  ordered = TRUE)
```

Select columns

select() retains named variables.
Selecting variables does not filter observations.

```
# |> passes the table to select(), which retains these two columns.
# head() previews six rows of the newly saved prices table.
prices <- restaurants |>
  select(restaurant_id, meal_price_usd)
head(prices)
```

Filter rows with AND

Keep Excellent restaurants with a meal price of at least USD 30.

```
# filter() keeps TRUE conditions; a comma combines conditions with AND.
# head() previews the result; nrow() counts its retained observations.
selected <- restaurants |>
  filter(quality_rating == "Excellent", meal_price_usd >= 30)
head(selected)
nrow(selected)
```

A condition evaluating to NA does not pass filter().

Membership and OR conditions

%in% tests membership in a set.

Use | when either of two conditions is enough.

```
# c() supplies allowed categories; %in% tests membership in that set.
# The comma also requires price < USD 20; head() previews six matches.
restaurants |>
  filter(quality_rating %in% c("Good", "Very Good"),
         meal_price_usd < 20) |>
  head()
```

A readable pipeline

Read the sequence from top to bottom.
Each result becomes the input to the next operation.

```
# filter() chooses rows and select() chooses columns.
# arrange() sorts; desc() reverses price order; head(5) retains five rows.
restaurants |>
  filter(meal_price_usd >= 30) |>
  select(restaurant_id, quality_rating, meal_price_usd) |>
  arrange(desc(meal_price_usd), restaurant_id) |>
  head(5)
```

dplyr also supports %>%; use a consistent pipe style.

Sorting and derived variables

arrange() sorts rows. mutate() creates or replaces columns.
Derived labels do not correct the original measurements.

```
# cut() uses the breakpoints in c(); right = FALSE gives [lower, upper) bands.
# mutate() adds columns; arrange() sorts rows; head() displays a preview.
prepared <- restaurants |>
  mutate(price_at_least_30 = meal_price_usd >= 30,
    price_band = cut(meal_price_usd,
      c(10, 20, 30, 40, 50), right = FALSE)) |>
  arrange(quality_rating, meal_price_usd)
head(prepared)
```

10 is included in [10,20); 50 falls outside these specified bands.

Count categories

count() counts observations in each category.

```
# |> passes the data to count(), which groups and counts category rows.
# name = "restaurants" names the frequency column in the returned table.
restaurants |>
  count(quality_rating, name = "restaurants")
```

Summarize within groups

group_by() defines groups for the following calculations.
summarise() returns the requested group statistics.

```
# n() counts rows; mean() averages prices; median() finds their middle value.
# group_by()/summarise() calculate by rating; .groups = "drop" ungroups the result.
group_summary <- restaurants |>
  group_by(quality_rating) |>
  summarise(n = n(),
    mean_price_usd = mean(meal_price_usd),
    median_price_usd = median(meal_price_usd),
    .groups = "drop")
group_summary
```

Validate totals and grouping

Check that group counts add to the original row count.
Grouping metadata affects later calculations.
ungroup() removes grouping explicitly when needed.

```
# sum() totals the saved counts; nrow() gives the original table size.
# stopifnot() stops on a failed condition and prints nothing when it passes.
stopifnot(sum(group_summary$n) == nrow(restaurants))
```

Summarize actual prices; avoid averaging arbitrary numeric category codes.

Missing values and duplicate identifiers

Check missingness before deciding which observations to omit.
Equal prices do not make two restaurant records duplicates.

```
# is.na() flags missing values; colSums() counts them in each column.
# anyDuplicated() returns 0 if none occur, or the first duplicate position.
colSums(is.na(restaurants))
anyDuplicated(restaurants$restaurant_id)
```

Use na.rm = TRUE only when the omitted observations and consequences are clear.

A lookup derived from the summary

Use the shared quality_rating key to attach group medians.

```
# select() retains the key and renames the median column for the lookup.
# left_join() attaches matching lookup rows; by specifies the key; head() previews it.
lookup <- group_summary |>
  select(quality_rating, group_median_usd = median_price_usd)
with_group_median <- restaurants |>
  left_join(lookup, by = "quality_rating")
head(with_group_median)
```

Join checks

A unique lookup key should preserve the 300 restaurant rows.
Repeated matching keys can multiply rows.
inner_join() keeps matches; left_join() retains left-table observations.

```
# nrow() compares row counts; anyNA() detects missing joined medians.
# ! negates the missingness result; stopifnot() halts if either check fails.
stopifnot(
  nrow(with_group_median) == nrow(restaurants),
  !anyNA(with_group_median$group_median_usd))
```

Reshape from wide to long

pivot_longer() moves selected columns into name/value rows.
The rating remains the identifier for each group.

```
# library() loads tidyr; select() retains the summary columns to reshape.
# pivot_longer() names new columns via names_to/values_to; -rating excludes the key.
library(tidyr)
long_summary <- group_summary |>
  select(quality_rating, mean_price_usd, median_price_usd) |>
  pivot_longer(-quality_rating,
    names_to = "measure", values_to = "price_usd")
long_summary
```

Reshape from long to wide

pivot_wider() restores measure columns when keys identify cells.

```
# pivot_wider() makes columns from values in the measure column.
# names_from chooses column names; values_from supplies their cell values.
long_summary |>
  pivot_wider(names_from = measure, values_from = price_usd)
```

Export a derived summary

Write report outputs to _output/, separate from the input CSV.
Run file-writing commands manually when an export is wanted.

```
# dir.create() prepares the folder; showWarnings = FALSE suppresses existing-folder warnings.
# write.csv() exports the table; row.names = FALSE omits an extra row-number column.
dir.create("_output", showWarnings = FALSE)
write.csv(group_summary,
  "_output/restaurant-price-summary.csv", row.names = FALSE)
```

Continue with the matching dplyr exercises in Quarto.

Lesson 4: Visualization with ggplot2

The prepared table now becomes input to a layered graphic.

The grammar of graphics

A plot combines data, mappings, and layers.
Build it gradually, checking what each added part changes.
The same framework supports many chart types.

Plot components

Data: the table to display.
Aesthetics: map variables to position, color, size, or shape.
Geometries: represent observations as points, lines, bars, and more.
Scales and coordinates control how values appear.

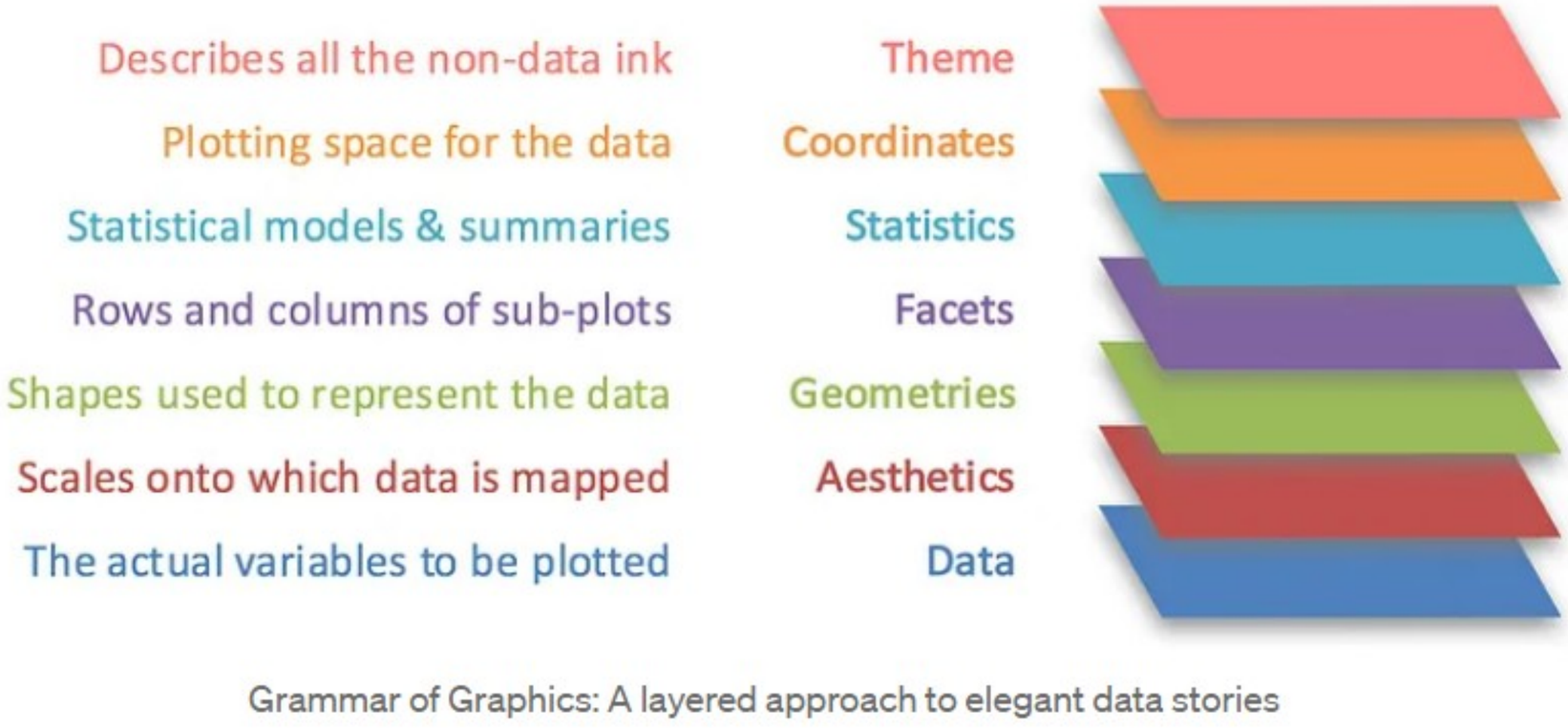
Additional plot components

Statistical transformations count, bin, smooth, or summarize.
Facets separate groups into panels.
Themes control nondata appearance such as fonts and backgrounds.

These components will appear in the examples that follow.

The lesson component illustration

Conceptual illustration from the Quarto lesson.



Source: existing lesson asset; original external attribution is not stated.

Layering illustration

The Quarto lesson also illustrates how graphical layers combine.



Static slide frame; open the Quarto lesson to view the animation.

Step 1: load the package and data

Use the lesson built-in mtcars dataset.

One row describes a car model.

```
# library() makes ggplot2 functions available in this R session.
# data() loads mtcars; mpg is miles per US gallon and wt is weight in 1,000 lb.
library(ggplot2)
data(mtcars)
```

Step 2: define data and mappings

ggplot() starts the plot.

aes() maps weight to x and fuel economy to y.

```
# ggplot() selects the table; aes() specifies the variable mappings.
# This defines the plot framework; a geometry is still needed to draw observations.
ggplot(data = mtcars, aes(x = wt, y = mpg))
```

Step 3: add a geometry

Use + to add a layer to a ggplot.

Different geometries represent different kinds of questions.

```
# ggplot()/aes() define mappings; geom_point() draws paired observations.
# geom_line() joins x order; factor() marks categories; geom_bar() counts rows.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_line()
ggplot(mtcars, aes(x = factor(cyl))) +
  geom_bar()
```

Connecting car models illustrates line syntax; it does not create a time sequence.

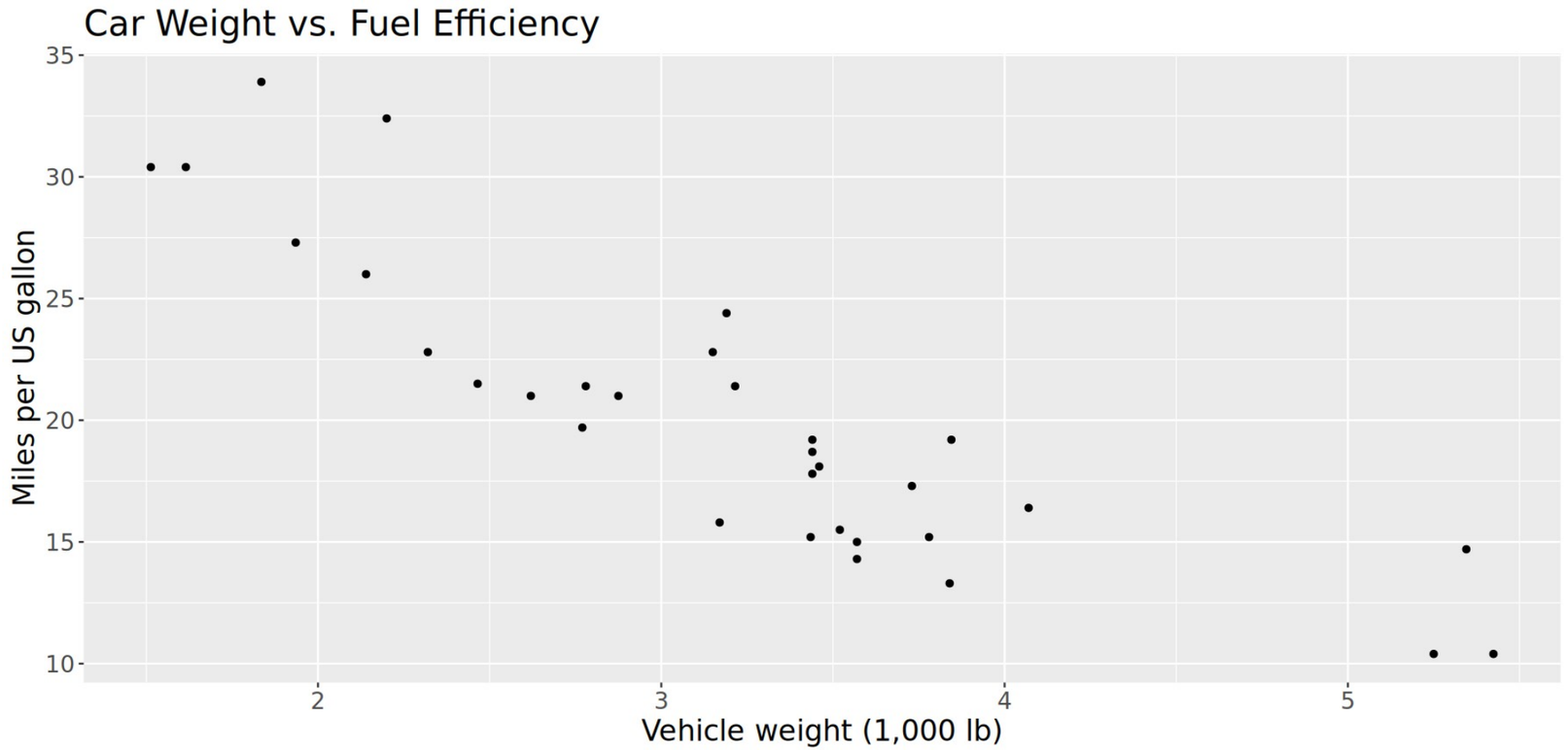
Step 4: add meaningful labels

labs() supplies chart and axis titles.
State measurement units explicitly.

```
# ggplot()/aes()/geom_point() draw the paired observations.
# labs() labels axes and the title; wt units are 1,000 lb and mpg uses US gallons.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon")
```

Step 4: add meaningful labels: result

labs() supplies chart and axis titles.
State measurement units explicitly.



Read the plotted result alongside the preceding code.

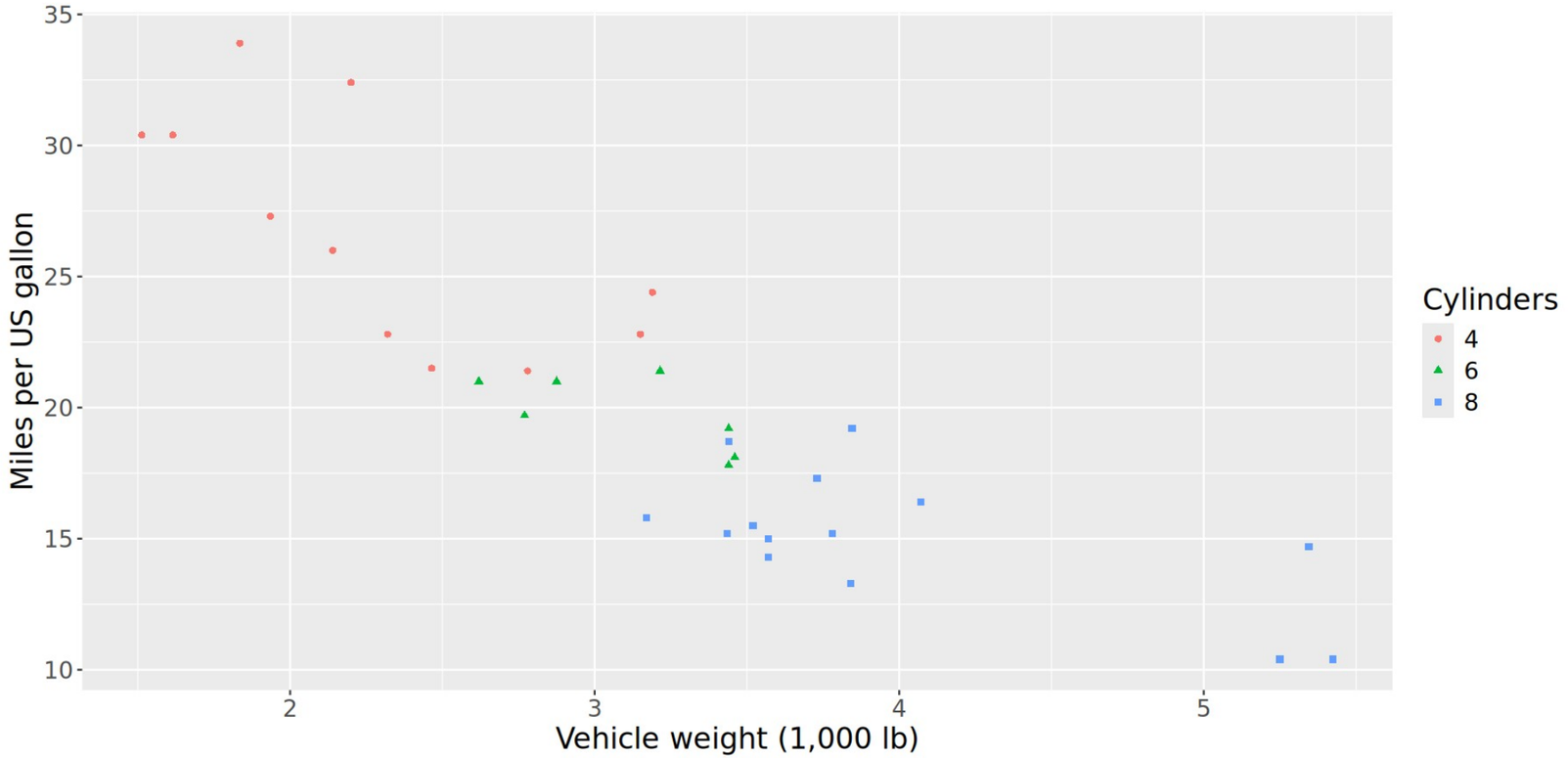
Step 5: map categories to appearance

Mapping inside aes() varies appearance by a variable.
Use color and shape together to distinguish cylinder groups.

```
# factor() treats cylinder counts as categories inside aes().
# geom_point() draws points; labs() names the color and shape legends.
ggplot(mtcars, aes(x = wt, y = mpg,
    color = factor(cyl), shape = factor(cyl))) +
  geom_point() +
  labs(color = "Number of Cylinders",
       shape = "Number of Cylinders")
```

Step 5: map categories to appearance: result

Mapping inside aes() varies appearance by a variable.
Use color and shape together to distinguish cylinder groups.



Read the plotted result alongside the preceding code.

Step 6: axis limits

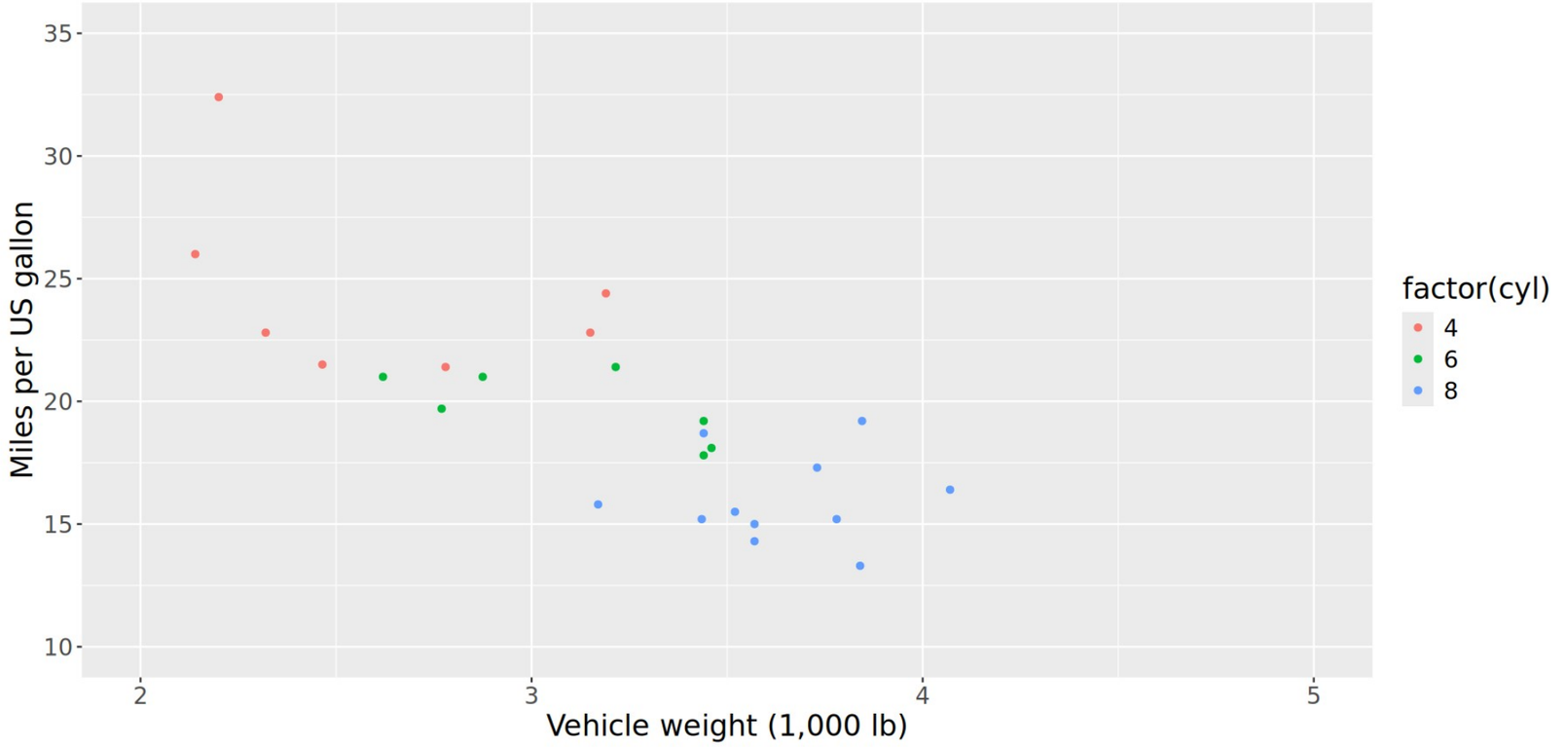
Scale limits can remove observations outside the displayed range.
Interpret the remaining data with this consequence in mind.

```
# ggplot()/aes() map data; factor() groups cylinders; geom_point() draws points.
# xlim()/ylim() set scale limits and can discard rows outside them.
ggplot(mtcars, aes(x = wt, y = mpg,
    color = factor(cyl))) +
  geom_point() +
  xlim(2, 5) +
  ylim(10, 35)
```

The shown ranges are weight 2,000–5,000 lb and fuel economy 10–35 mpg.

Step 6: axis limits: result

Scale limits can remove observations outside the displayed range. Interpret the remaining data with this consequence in mind.



Read the plotted result alongside the preceding code.

Step 6: scale labels and breaks

Scales control axis names, breaks, and other display choices.

```
# ggplot()/aes()/geom_point() create the scatterplot.
# scale_*_continuous() customizes axes; seq(..., by = 0.5) requests spaced ticks.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  scale_x_continuous(name = "Weight (1000 lbs)",
    breaks = seq(2, 5, by = 0.5)) +
  scale_y_continuous(name = "Miles per Gallon")
```

Step 7: adjust the legend

A legend explains a mapped visual property.

```
# factor()/aes() map cylinder categories; geom_point() draws observations.
# labs() names mappings; guides()/guide_legend() customize the color legend.
ggplot(mtcars, aes(x = wt, y = mpg,
    color = factor(cyl))) +
  geom_point() +
  labs(color = "Number of Cylinders") +
  guides(color = guide_legend(title = "Cylinders"))
```

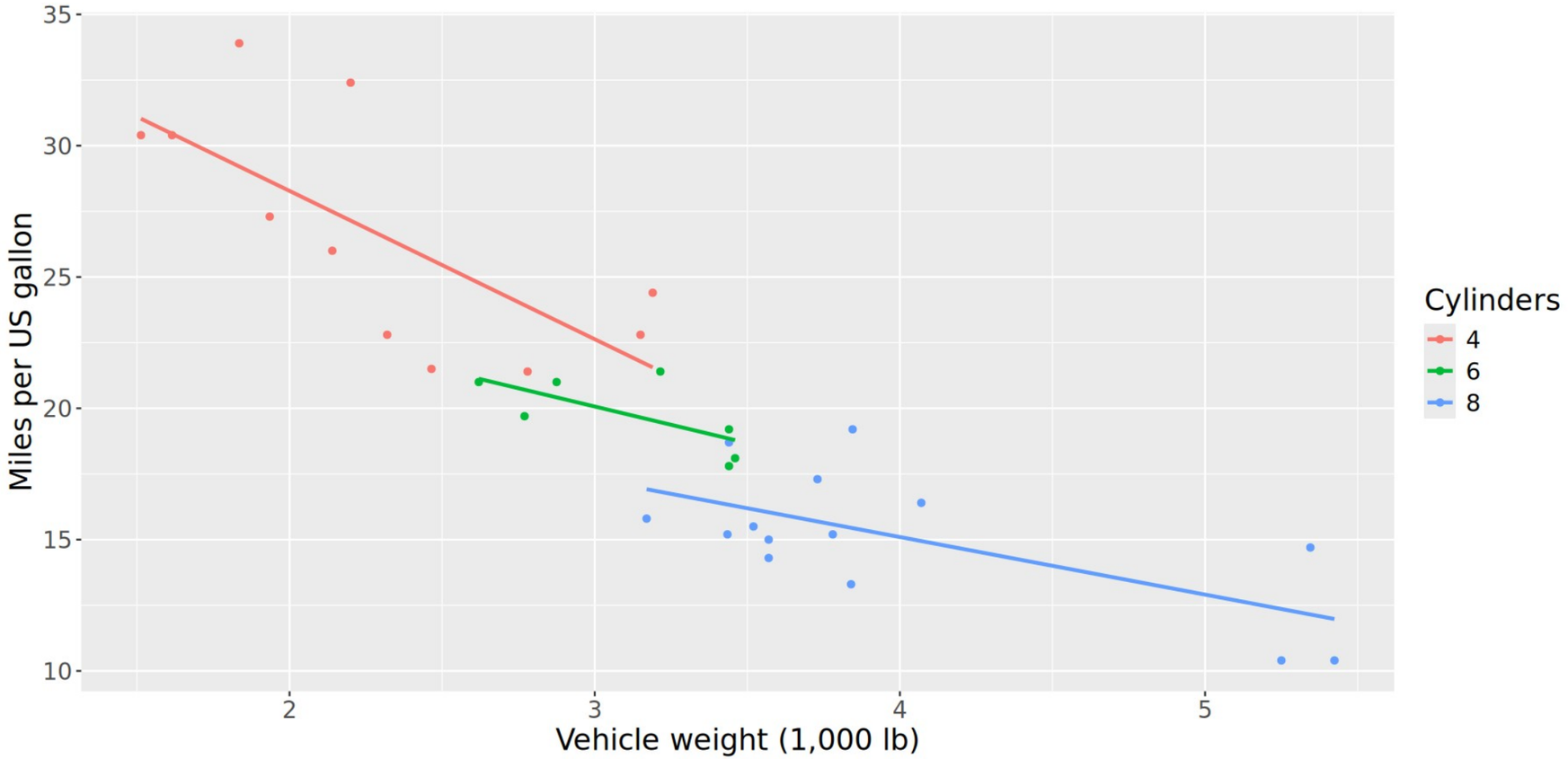
Step 8: a fitted line for each group

A global color mapping also groups the fitted lines.

```
# ggplot()/aes()/factor() map cylinder groups to all inherited layers.
# geom_smooth(method = "lm") fits lines; se = FALSE hides mean-trend bands.
ggplot(mtcars, aes(x = wt, y = mpg,
    color = factor(cyl))) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

Step 8: a fitted line for each group: result

A global color mapping also groups the fitted lines.



Read the plotted result alongside the preceding code.

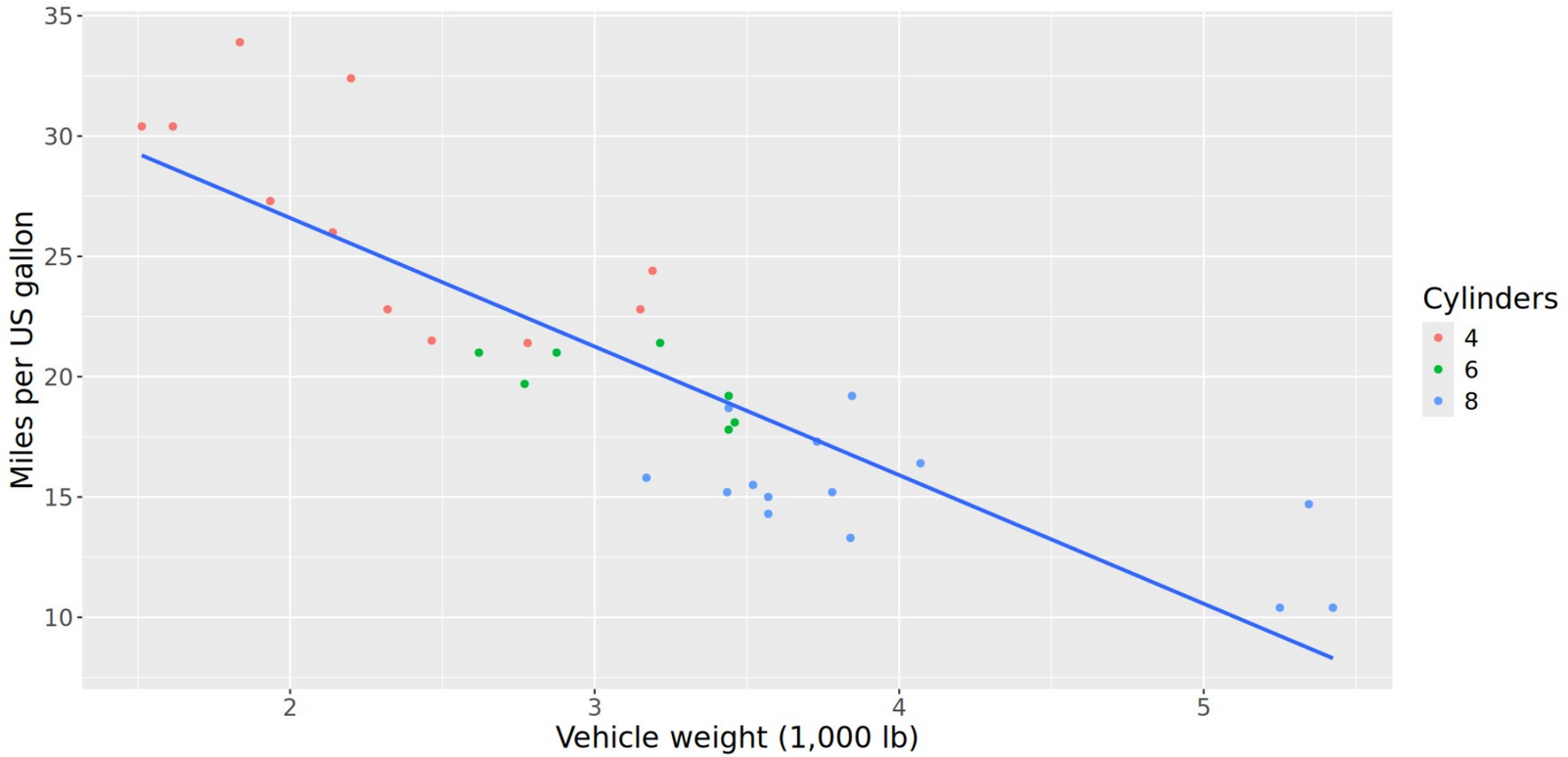
Step 8: one fitted line for all observations

Move the color mapping into the points layer.
The fitted line then uses the combined observations.

```
# aes()/factor() inside geom_point() color only the points by cylinder group.
# geom_smooth() inherits x/y only; lm fits one line and se = FALSE hides its band.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = factor(cyl))) +
  geom_smooth(method = "lm", se = FALSE)
```

Step 8: one fitted line for all observations: result

Move the color mapping into the points layer.
The fitted line then uses the combined observations.



Read the plotted result alongside the preceding code.

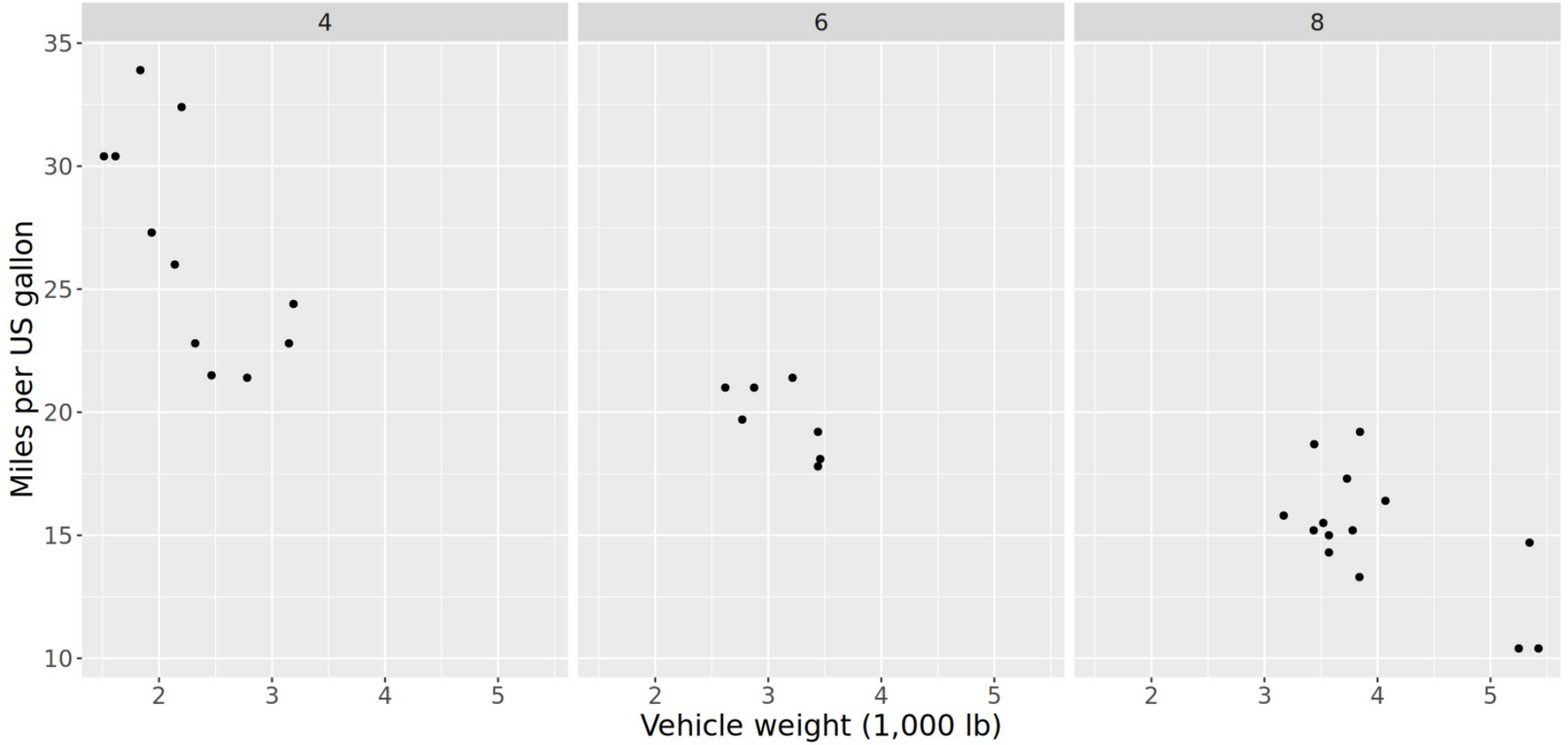
Faceting with facet_wrap()

Split the observations into panels by cylinder count.
Compare the same variables across groups.

```
# ggplot()/aes()/geom_point() define the same scatterplot in each panel.
# facet_wrap(~ cyl) makes one panel per distinct cylinder count.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_wrap(~ cyl)
```

Faceting with facet_wrap(): result

Split the observations into panels by cylinder count.
Compare the same variables across groups.



Read the plotted result alongside the preceding code.

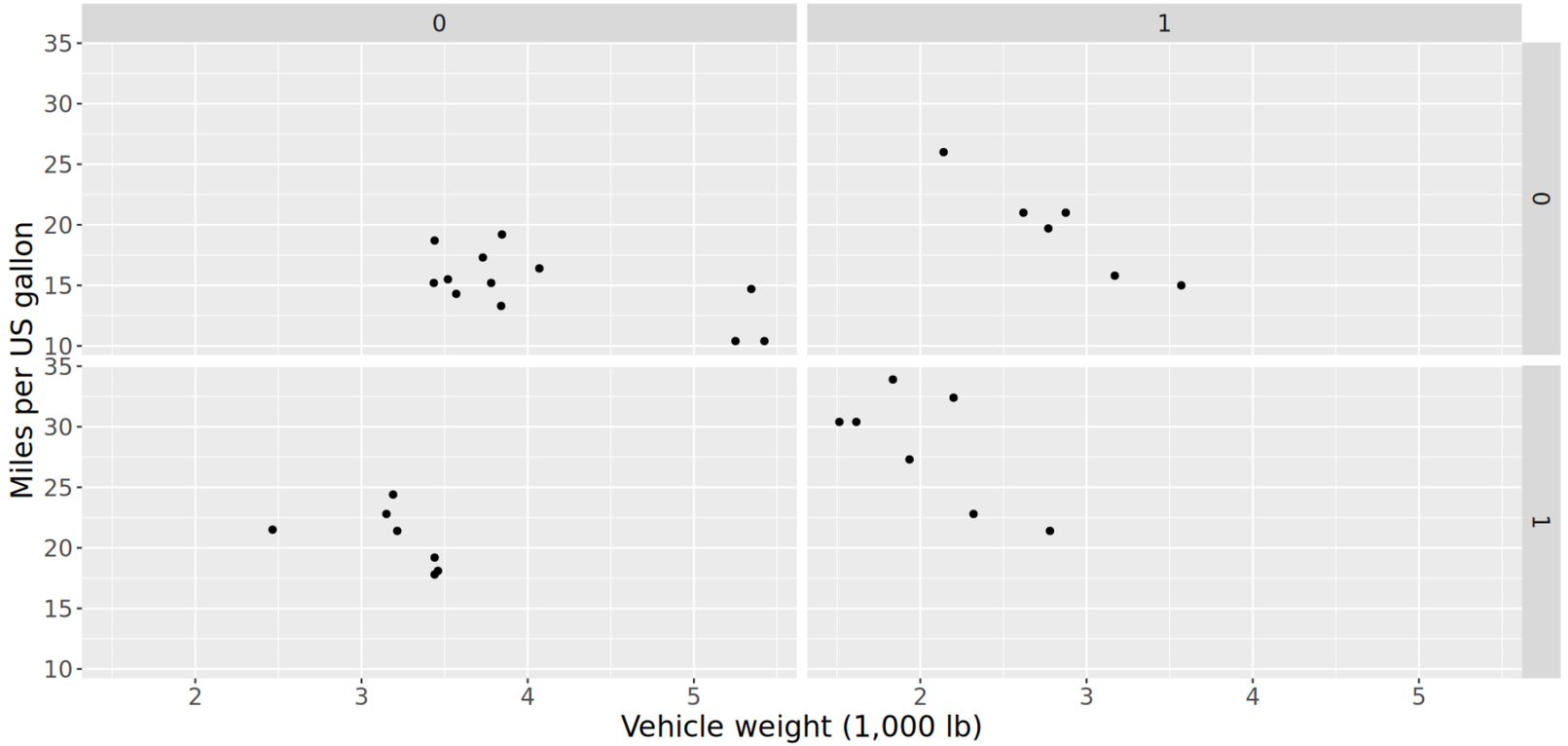
Faceting with facet_grid()

Use engine type for panel rows and transmission for columns.
Panel comparisons require attention to scales.

```
# ggplot()/aes()/geom_point() draw the observations in each panel.
# facet_grid(vs ~ am) puts engine type in rows and transmission in columns.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_grid(vs ~ am)
```

Faceting with facet_grid(): result

Use engine type for panel rows and transmission for columns.
Panel comparisons require attention to scales.



Read the plotted result alongside the preceding code.

Geometry examples

Keep the same plotting grammar and change the geometry.

The following examples follow the Quarto order.

1. Scatter plot

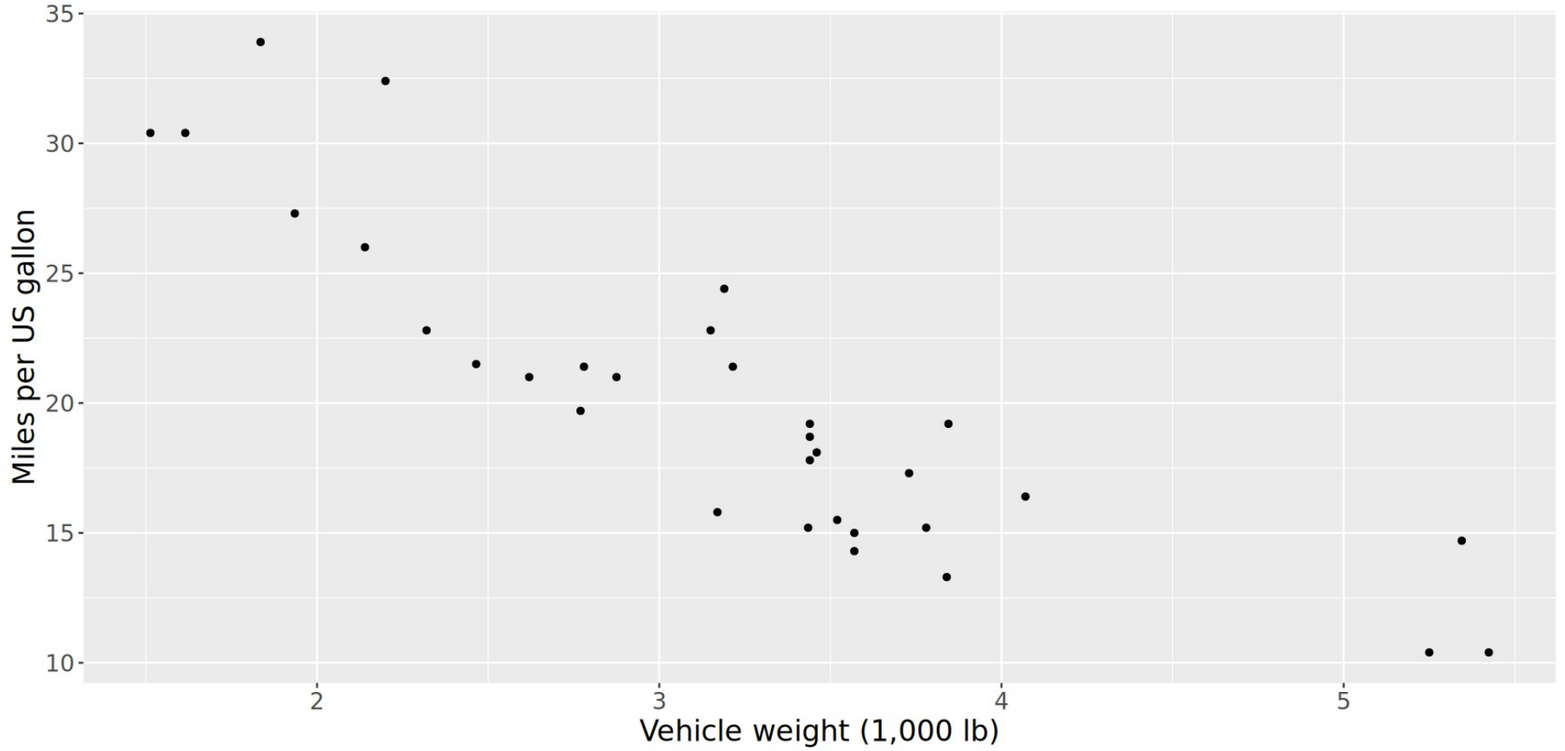
Numerical x and y. Each point is one observation.
Look for direction, clusters, unusual points, and curvature.

```
# ggplot() chooses data; aes() maps weight and fuel economy to x and y.
# geom_point() draws one point for each row.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```

1. Scatter plot: example

Business use: advertising and sales; customer age and preference.

استخدام تجاري: الإعلان والمبيعات؛ وعمر العميل وتفضيلاته.



Same example data and geometry as the Quarto lesson.

2. Line plot

Use ordered x with numerical y.

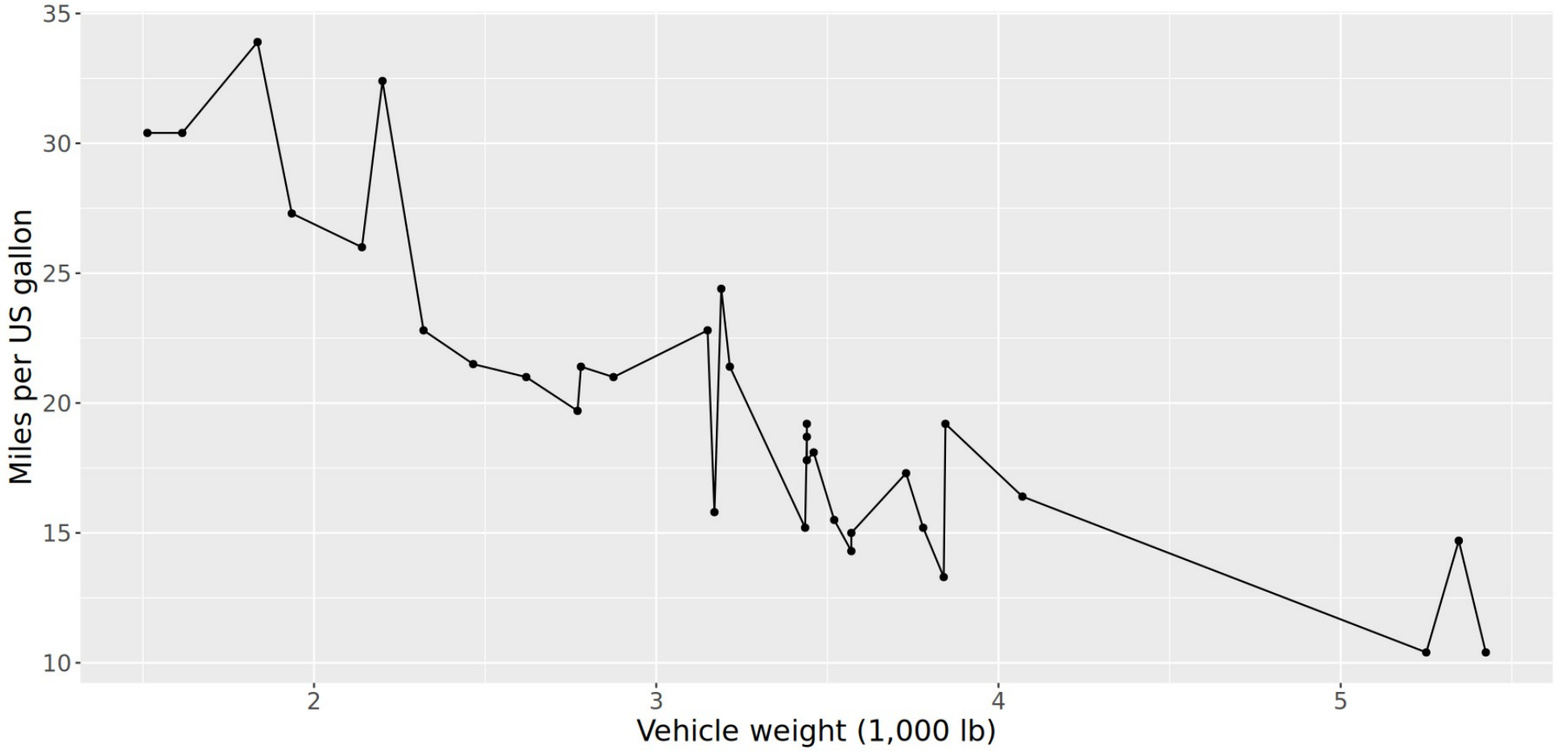
The lesson connects cars in weight order, which is not a time series.

```
# ggplot()/aes() map the two variables; geom_point() shows original observations.
# geom_line() joins observations in x order within groups.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_line() + geom_point()
```

2. Line plot: example

Business use: revenue, stock prices, or website traffic over time.

استخدام تجاري: الإيرادات أو أسعار الأسهم أو زيارات الموقع عبر الزمن.



Same example data and geometry as the Quarto lesson.

3. Fitted trend

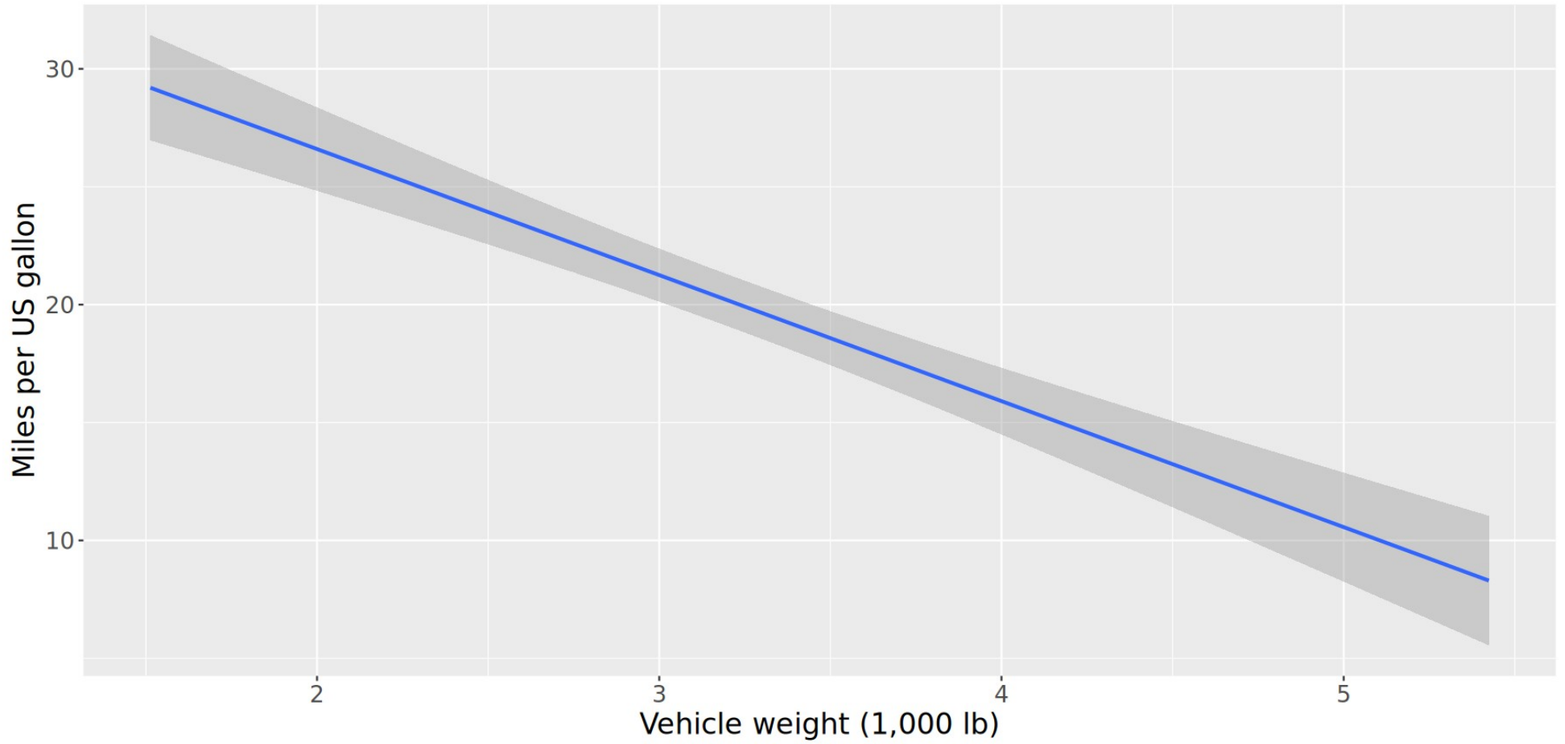
Numerical x and y. A fitted trend summarizes a relationship.
Outliers remain relevant to the fit and interpretation.

```
# ggplot()/aes() define the paired data; geom_smooth() adds a fitted trend.
# method = "lm" fits a line; the default band concerns the estimated mean trend.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_smooth(method = "lm")
```

3. Fitted trend: example

Business use: explore broad trends in fluctuating financial or sales data.

استخدام تجاري: استكشاف الاتجاهات العامة في البيانات المالية أو بيانات المبيعات المتقلبة.



Same example data and geometry as the Quarto lesson.

4. Category counts

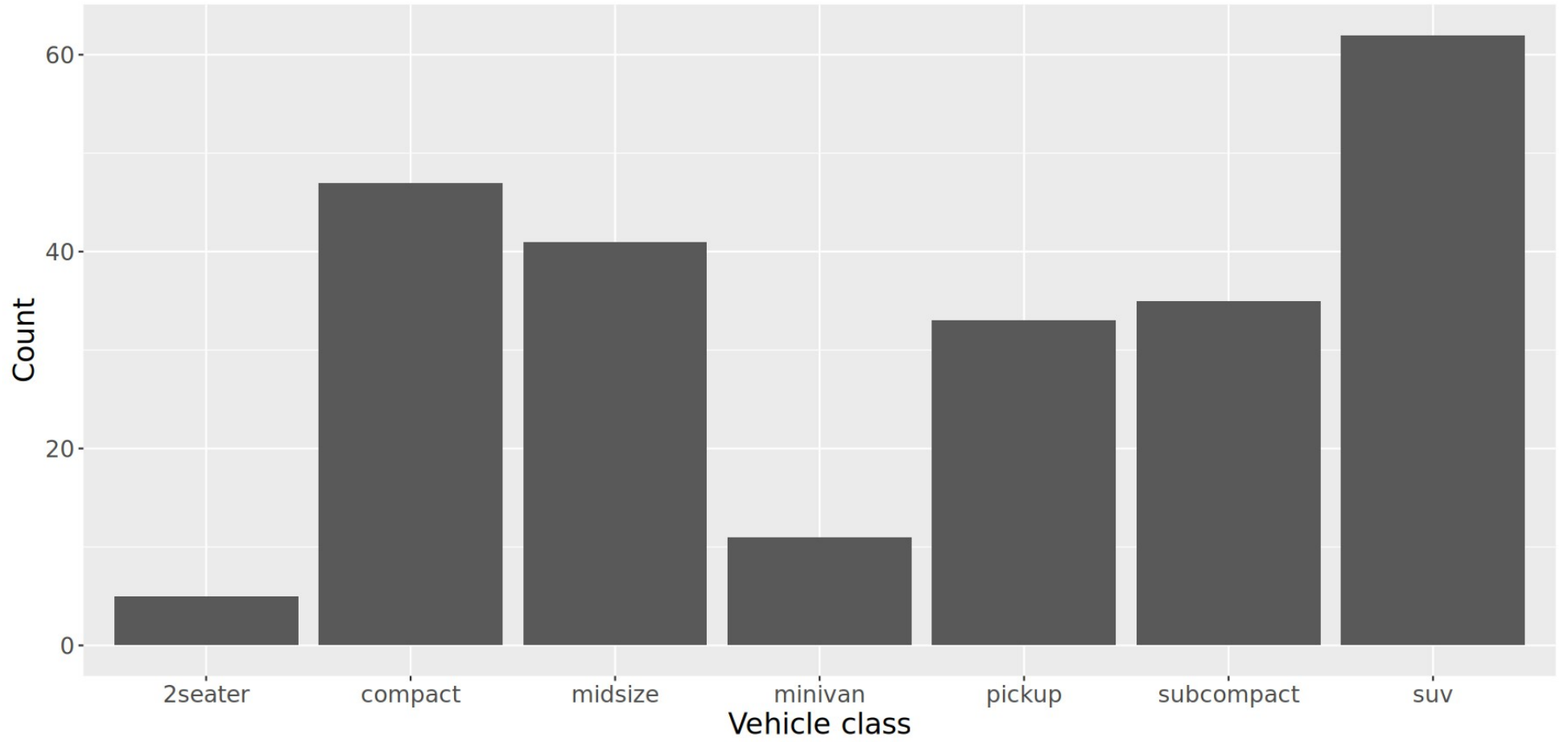
Categorical x. Bar height counts observations by default.
For precomputed quantities, use geom_col().

```
# ggplot()/aes() map the vehicle class to x.
# geom_bar() counts rows in each category by default.
ggplot(mpg, aes(x = class)) +
  geom_bar()
```

4. Category counts: example

Business use: compare product categories, customers by region, or departments.

استخدام تجاري: مقارنة فئات المنتجات أو العملاء حسب المنطقة أو الأقسام.



Same example data and geometry as the Quarto lesson.

5. Numerical distribution

Numerical x is divided into bins.

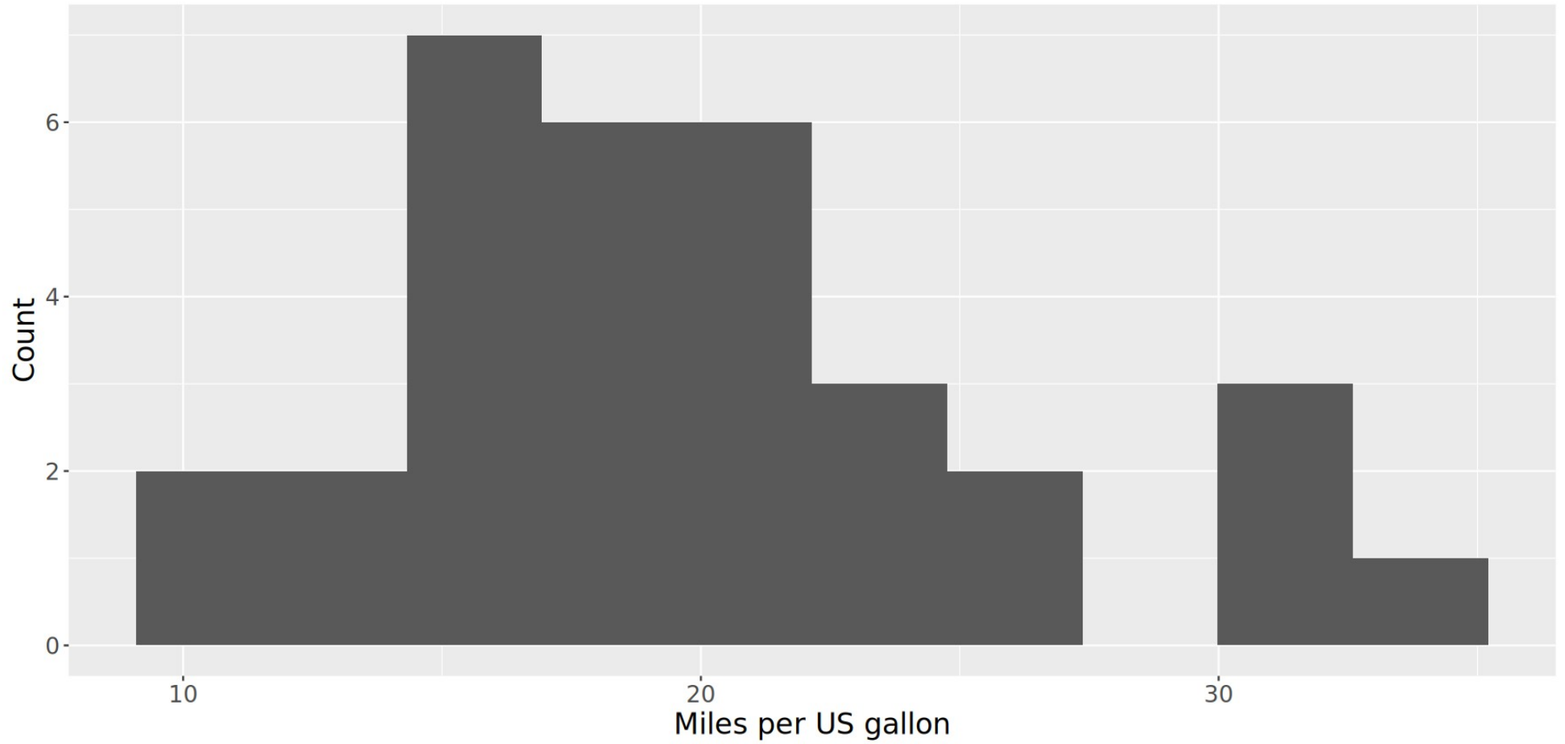
Bin choices affect the apparent shape.

```
# ggplot()/aes() map fuel economy; geom_histogram() counts observations in bins.
# bins = 10 requests ten bins; compare other bin choices before interpreting shape.
ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(bins = 10)
```

5. Numerical distribution: example

Business use: distributions of transaction sizes, customer ages, or response times.

استخدام تجاري: توزيعات قيم المعاملات أو أعمار العملاء أو أزمنة الاستجابة.



Same example data and geometry as the Quarto lesson.

6. Grouped boxplots

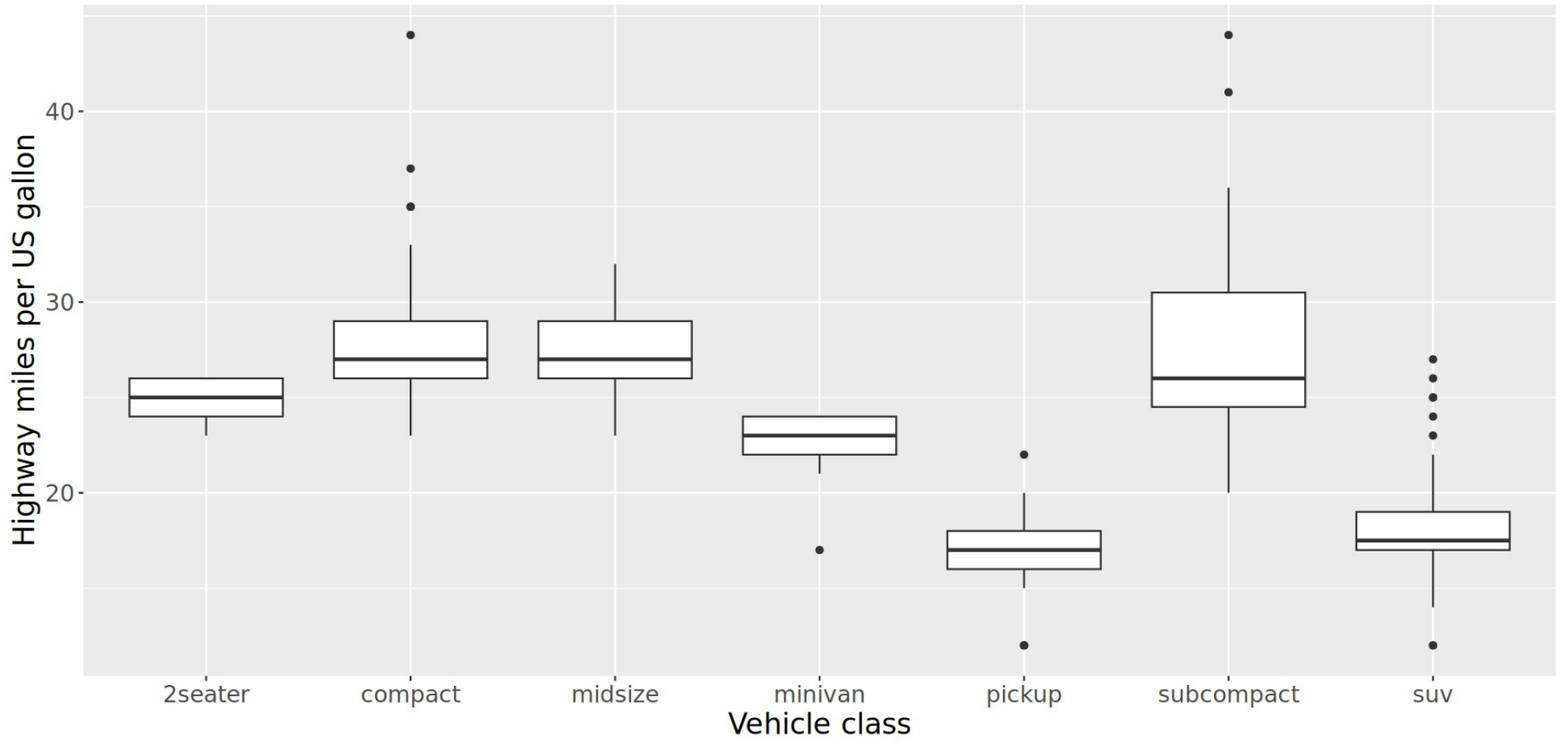
Compare a numerical distribution across categories.
Median and quartiles summarize the center and spread.
Beyond-whisker points flag observations for inspection.

```
# ggplot()/aes() map class groups and highway miles per US gallon.
# geom_boxplot() summarizes each group and marks beyond-whisker observations.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot()
```

6. Grouped boxplots: example

Business use: compare satisfaction by branch or sales distributions by region.

استخدام تجاري: مقارنة الرضا حسب الفرع أو توزيعات المبيعات حسب المنطقة.



Same example data and geometry as the Quarto lesson.

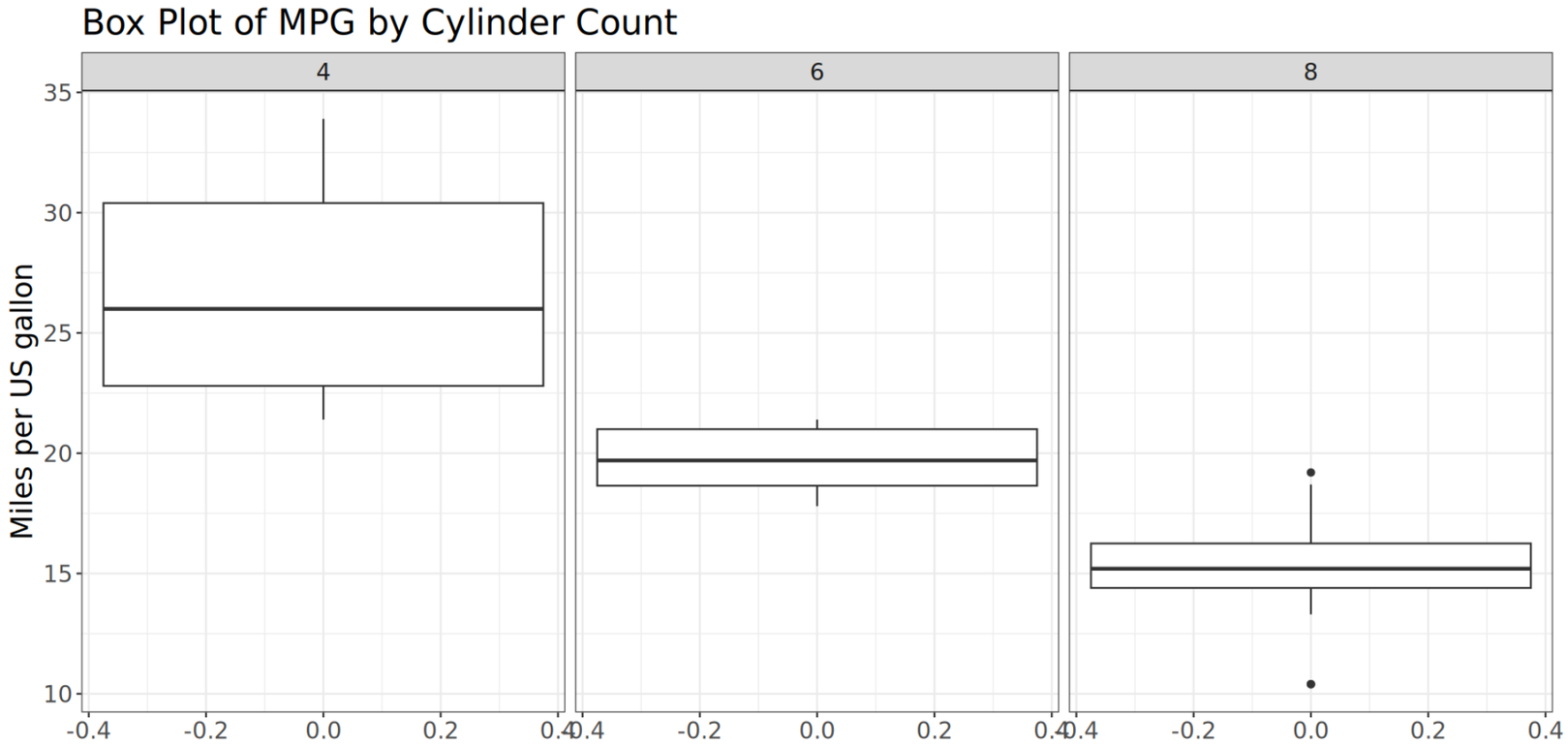
Boxplot variants in the lesson

Compare cylinder groups directly or use one facet per group.
theme_bw() applies a white background with reference grid lines.

```
# ggplot()/aes()/factor() define groupings; geom_boxplot() summarizes distributions.
# facet_wrap() splits groups into panels; theme_bw() changes nondata appearance.
ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot() + theme_bw()
ggplot(mtcars, aes(y = mpg)) +
  geom_boxplot() + facet_wrap(~ cyl) + theme_bw()
```

Boxplot variants in the lesson: result

Compare cylinder groups directly or use one facet per group.
theme_bw() applies a white background with reference grid lines.



Read the plotted result alongside the preceding code.

7. Area plot

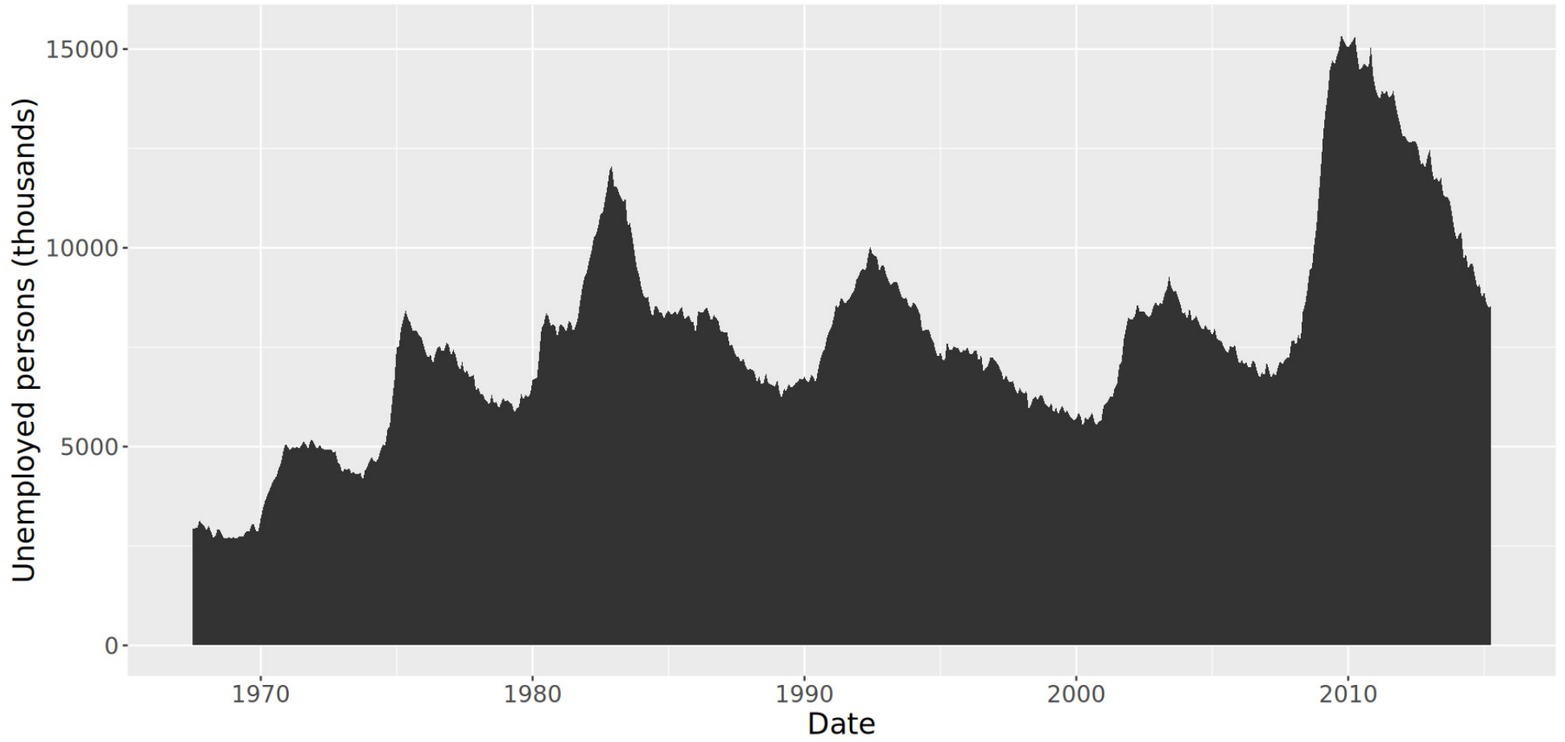
Fill the area below a sequence of numerical values.
The example shows unemployment levels over time, not a rate.

```
# ggplot()/aes() map date and unemployment counts from economics.
# geom_area() fills beneath the line; unemploy is measured in thousands of persons.
ggplot(economics, aes(x = date, y = unemploy)) +
  geom_area()
```

7. Area plot: example

Business use: show cumulative sales or contributions of product categories over time.

استخدام تجاري: عرض المبيعات التراكمية أو مساهمات فئات المنتجات عبر الزمن.



Same example data and geometry as the Quarto lesson.

8. Density plot

A smooth estimate of a numerical distribution.

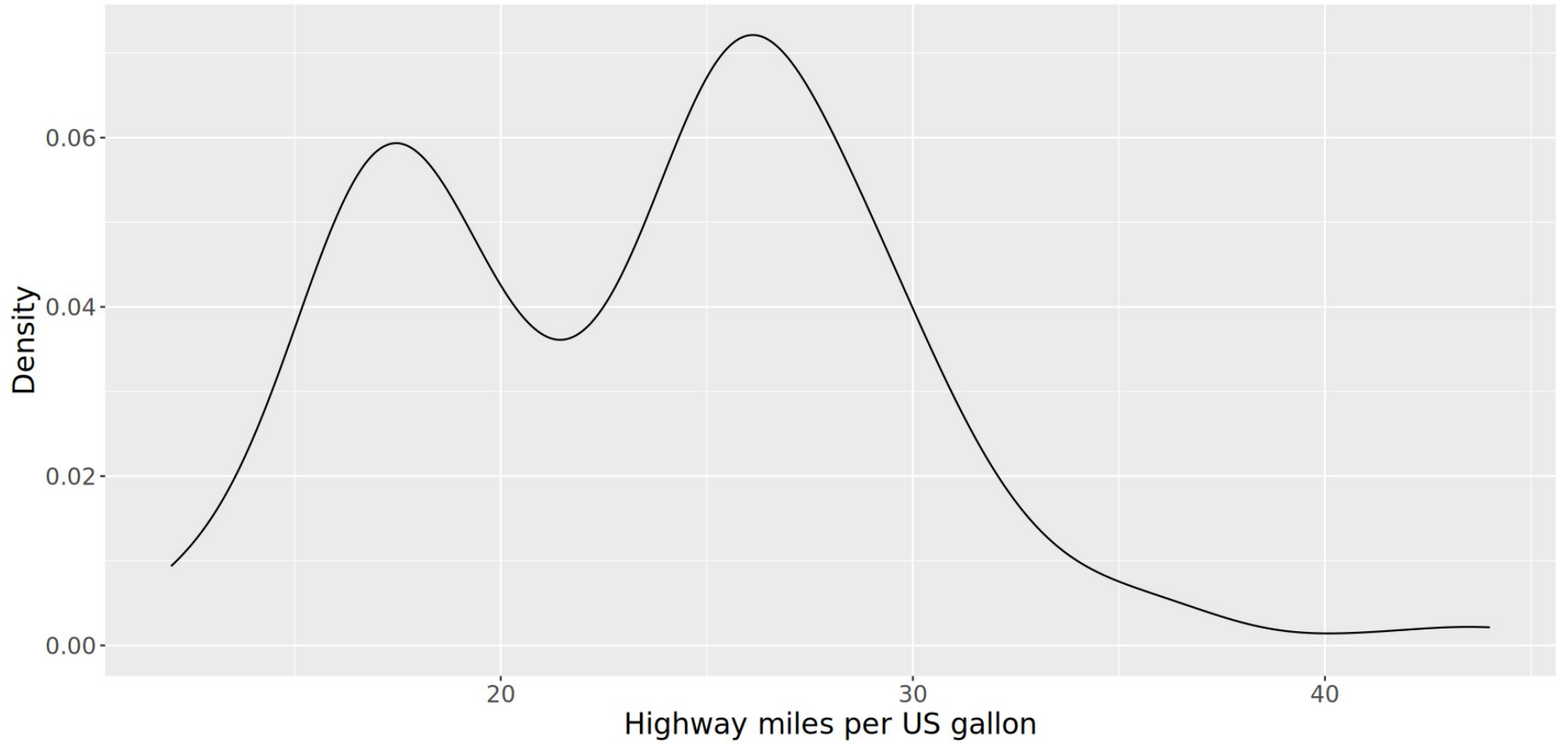
The curve shows density rather than raw counts.

```
# ggplot()/aes() map highway miles per US gallon.
# geom_density() estimates a density curve; its smoothing choice affects shape.
ggplot(mpg, aes(x = hwy)) +
  geom_density()
```

8. Density plot: example

Business use: compare pricing patterns or income distributions in customer segments.

استخدام تجاري: مقارنة أنماط الأسعار أو توزيعات الدخل في شرائح العملاء.



Same example data and geometry as the Quarto lesson.

9. Violin plot

Compare density shapes within categories.
A plain violin does not automatically draw boxplot summaries.

```
# ggplot()/aes() map class and highway fuel economy.
# geom_violin() draws a mirrored density for each category.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_violin()
```

9. Violin plot: example

Business use: compare customer satisfaction or product ratings across categories.

استخدام تجاري: مقارنة رضا العملاء أو تقييمات المنتجات بين الفئات.



Same example data and geometry as the Quarto lesson.

10. Columns for supplied values

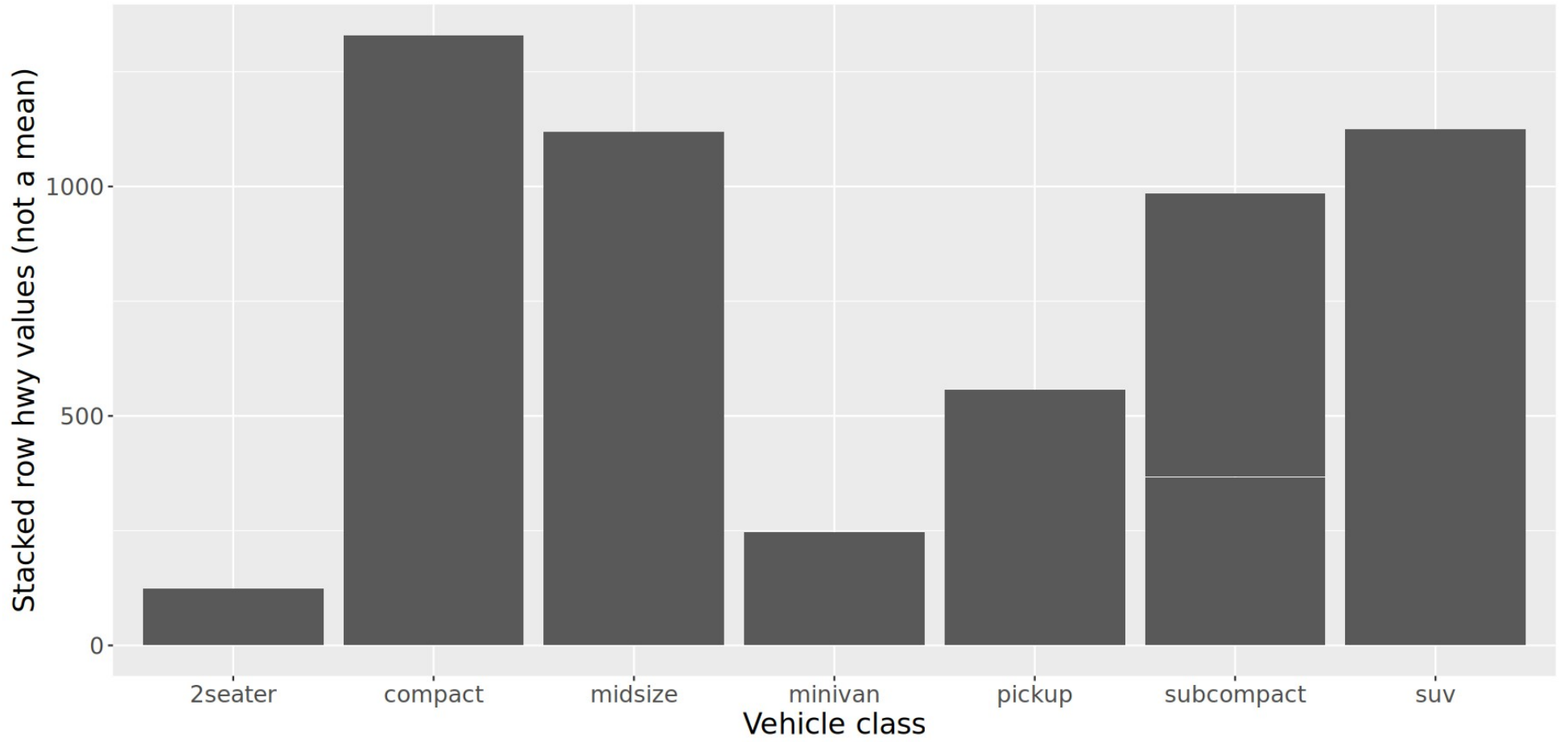
geom_col() uses y values as supplied.
Repeated categories stack by default. It does not calculate a mean.

```
# ggplot()/aes() map the original row values to categories and y.
# geom_col() uses those values directly and stacks repeated class rows by default.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_col()
```

10. Columns for supplied values: example

Business use: compare monthly revenue, category sales, or inventory levels.

استخدام تجاري: مقارنة الإيرادات الشهرية أو مبيعات الفئات أو مستويات المخزون.



Same example data and geometry as the Quarto lesson.

Averages require a summary first

The original geom_col() example stacks row values.
To display means, calculate one mean per class before plotting.
This distinction matters when interpreting a column chart.

The example is retained; the original description of its heights as averages is corrected.

11. Heatmap

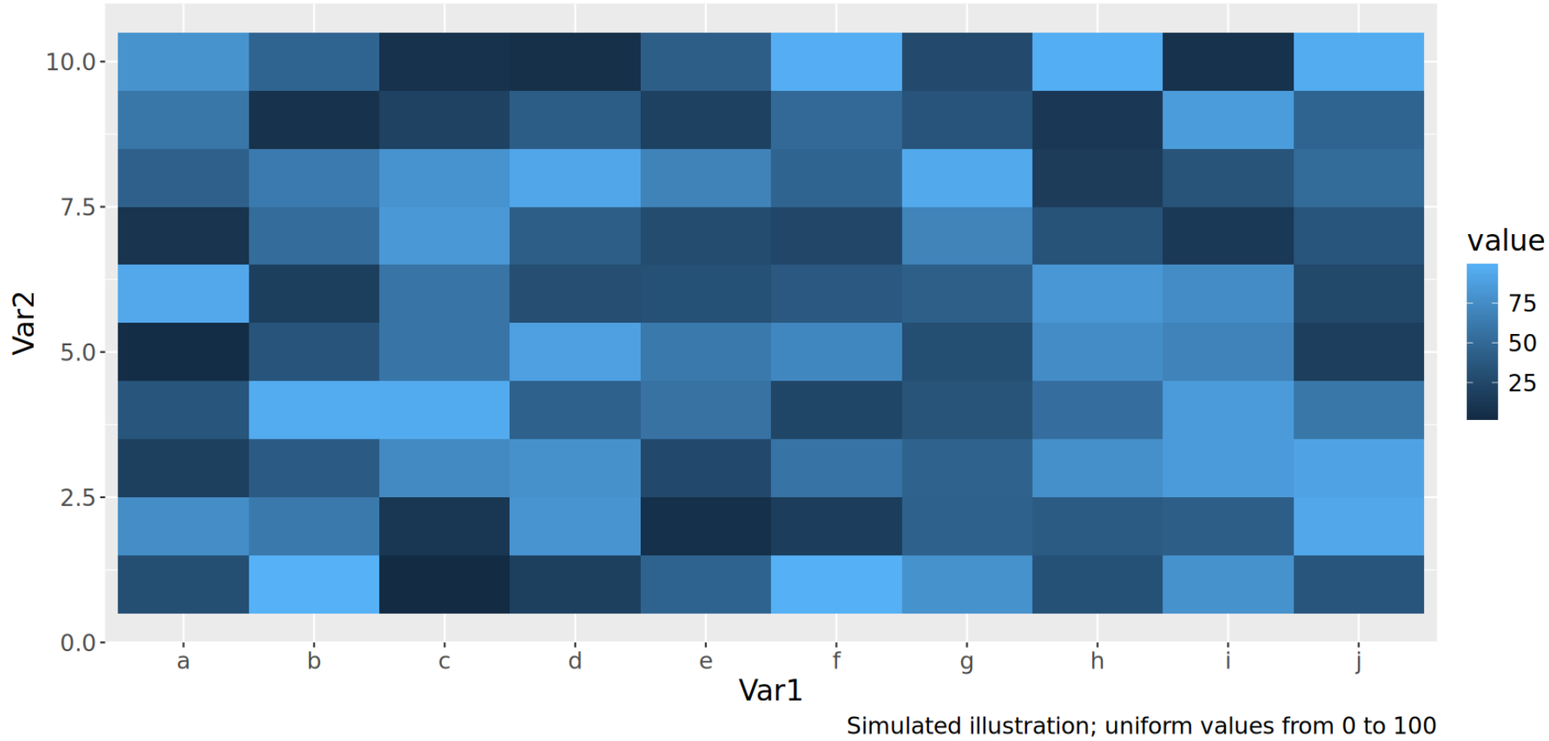
Two grid dimensions identify cells; fill encodes a third value.
These are simulated values rather than observed business data.

```
# expand.grid() creates combinations; runif() simulates 100 uniform values from 0 to 100.
# ggplot()/aes() map the cells and fill; geom_tile() draws the rectangular cells.
data <- expand.grid(Var1 = letters[1:10], Var2 = 1:10)
data$value <- runif(100, min = 0, max = 100)
ggplot(data, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile()
```

11. Heatmap: example

Business use: day/hour traffic, region/product sales, or a correlation matrix.

استخدام تجاري: الزيارات حسب اليوم والساعة أو المبيعات حسب المنطقة والمنتج أو مصفوفة الارتباط.



Same example data and geometry as the Quarto lesson.

12. Text and labels

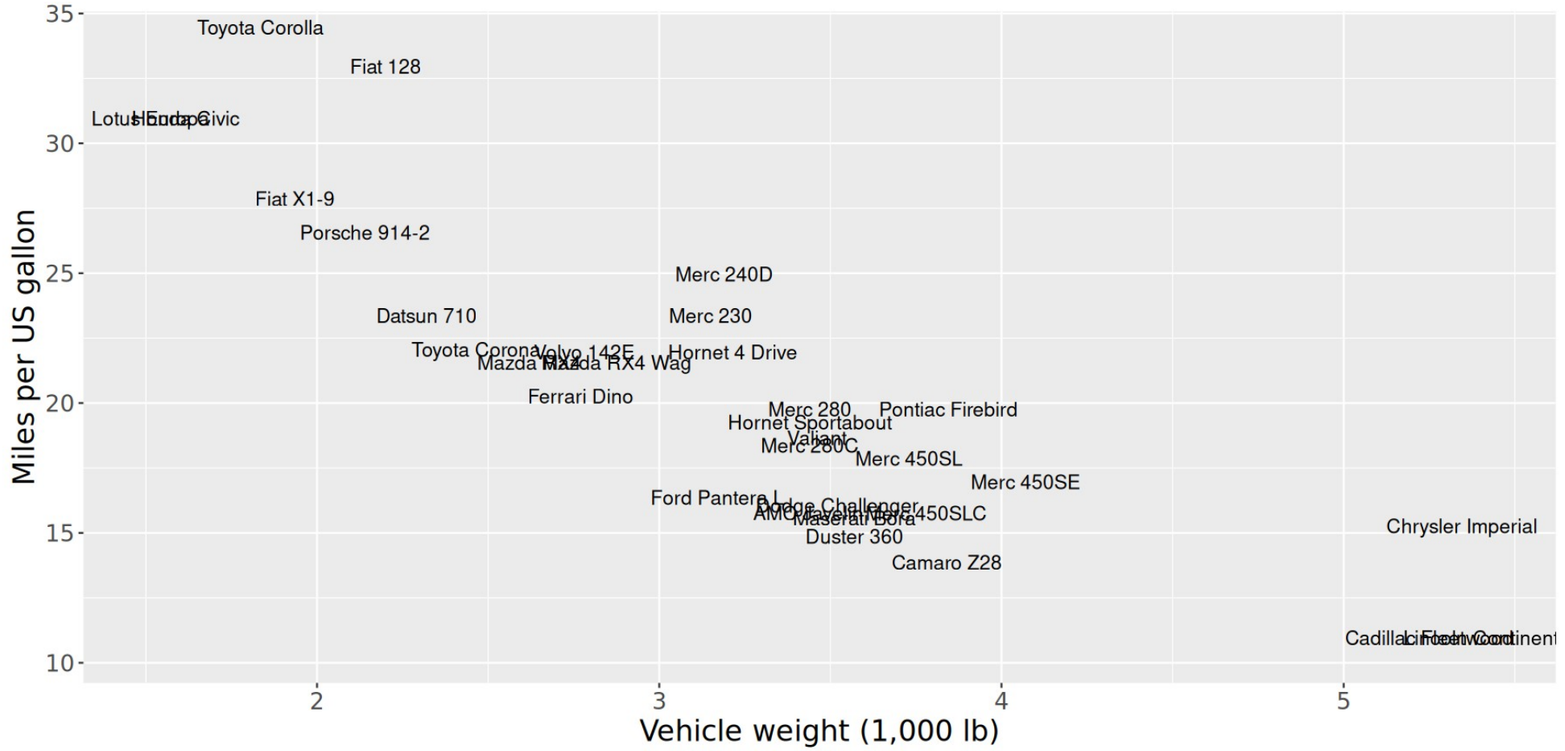
Identify observations with their row labels.
geom_label() adds a background behind the text.

```
# rownames() provides model labels; ggplot()/aes() map labels and coordinates.
# geom_text()/geom_label() add text; vjust = -0.5 moves it above its anchor.
ggplot(mtcars, aes(x = wt, y = mpg,
    label = rownames(mtcars))) +
  geom_text(vjust = -0.5)
ggplot(mtcars, aes(x = wt, y = mpg,
    label = rownames(mtcars))) +
  geom_label(vjust = -0.5)
```

12. Text and labels: example

Business use: label unusual points, important observations, or chart totals.

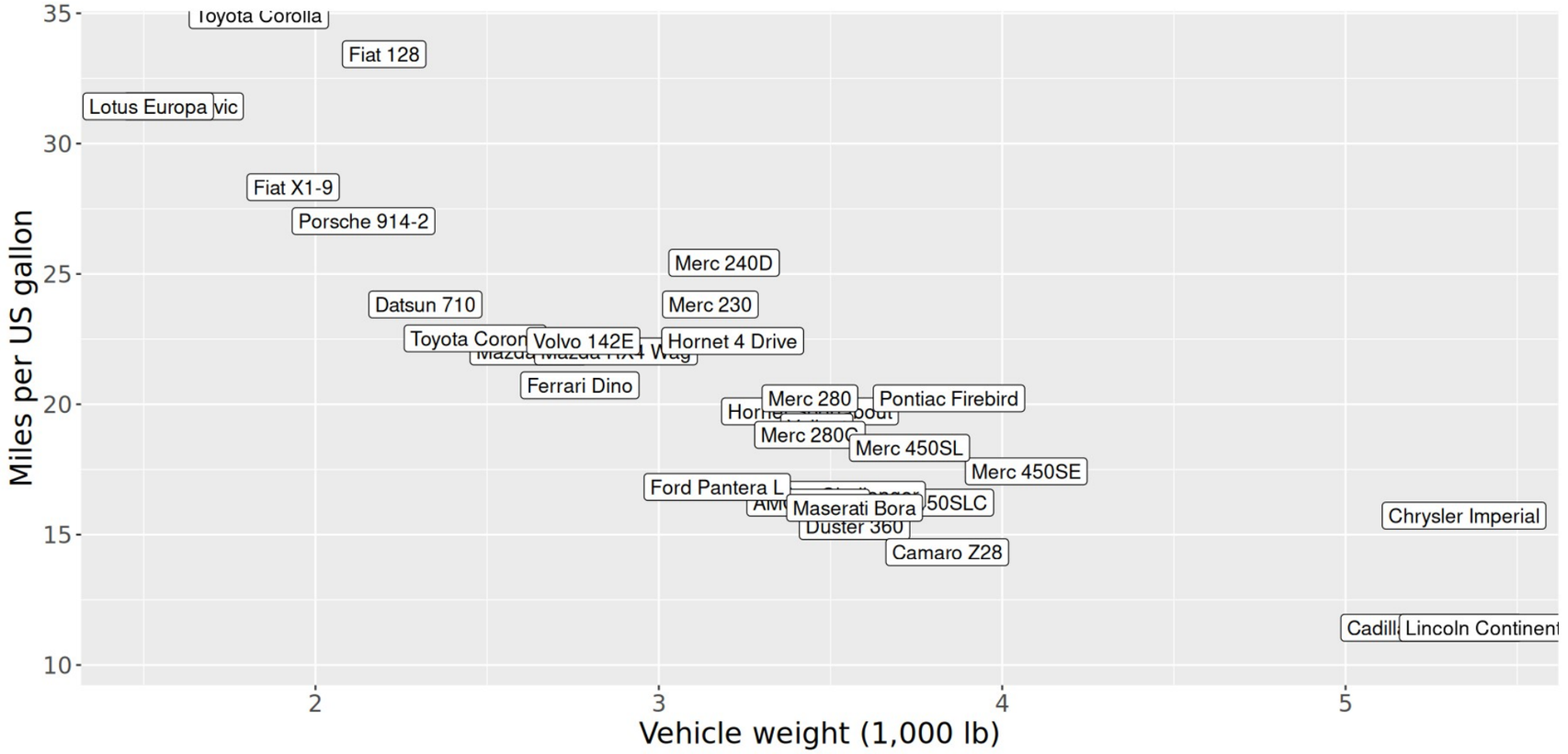
استخدام تجاري: تسمية النقاط غير المعتادة أو المشاهدات المهمة أو المجاميع على الرسم.



Same example data and geometry as the Quarto lesson.

Label backgrounds

The label geometry adds a background for readability.
Many labels can still overlap.



13. Ribbon plot

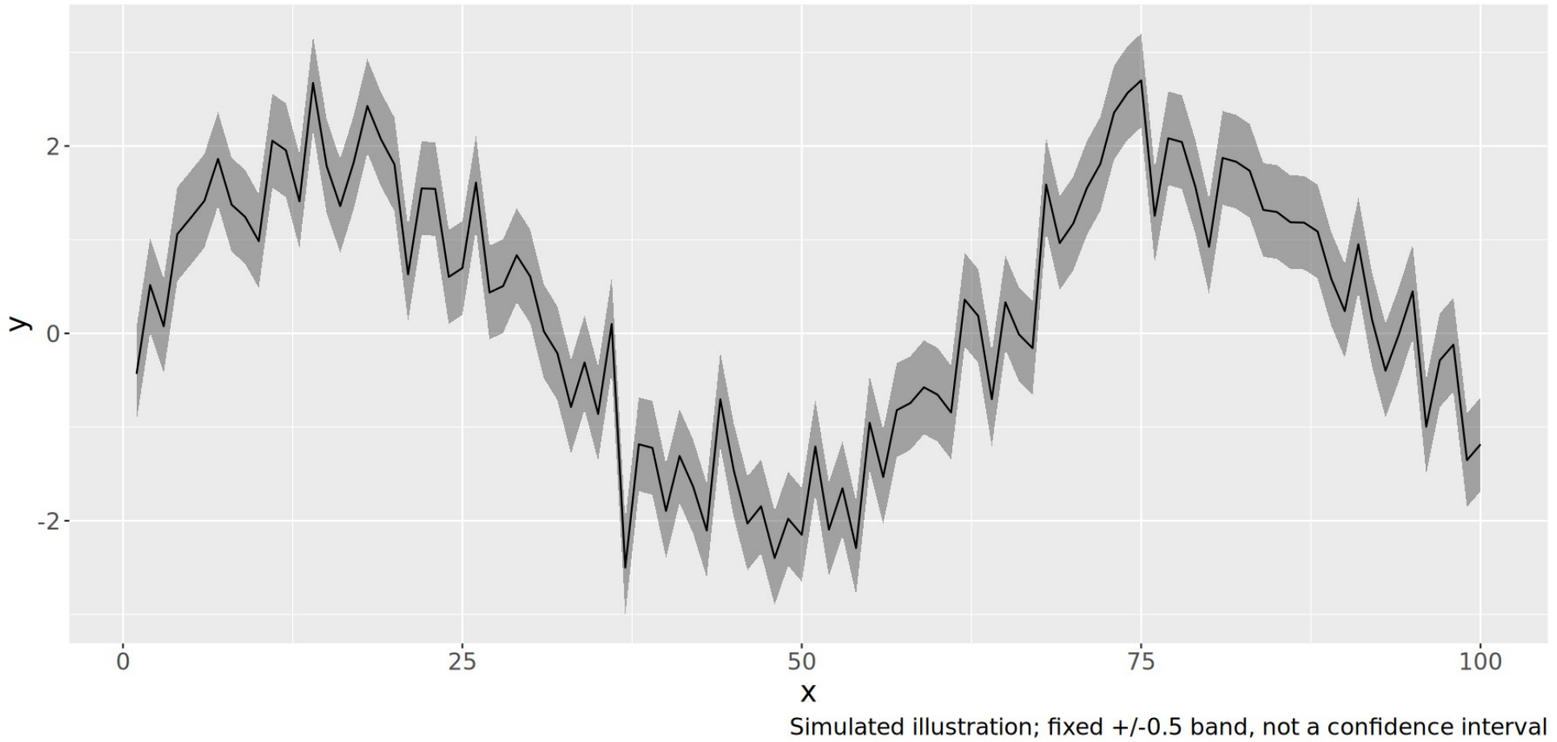
A ribbon fills the interval between ymin and ymax.
The lesson uses an artificial sine trend plus random noise.
Its fixed ±0.5 band is illustrative, not a calculated confidence interval.

```
# data.frame() creates x; sin() and rnorm() create an artificial noisy trend.
# aes() maps bounds; geom_ribbon() fills them with alpha transparency; geom_line() adds y.
df <- data.frame(x = 1:100)
df$y <- sin(df$x / 10) * 2 + rnorm(100, sd = 0.5)
df$ymax <- df$y + 0.5
df$ymin <- df$y - 0.5
ggplot(df, aes(x = x, ymin = ymin, ymax = ymax, y = y)) +
  geom_ribbon(alpha = 0.4) + geom_line()
```

13. Ribbon plot: example

Business use: display forecast ranges or statistically calculated uncertainty bands.

استخدام تجاري: عرض نطاقات التنبؤ أو نطاقات عدم اليقين المحسوبة إحصائيًا.



Same example data and geometry as the Quarto lesson.

14. Jittered points

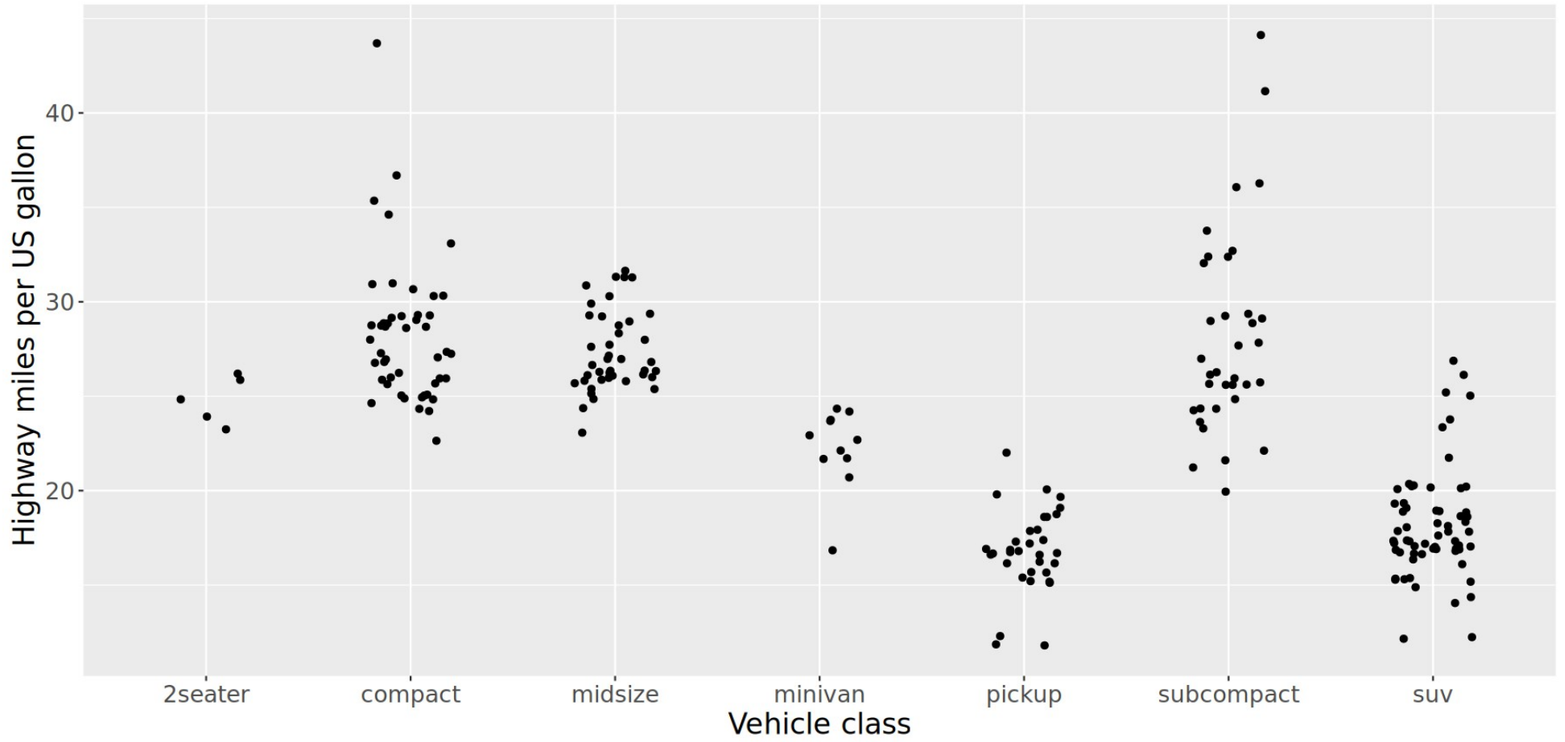
Small display offsets reveal overlapping observations.
The source measurements remain unchanged.

```
# ggplot()/aes() map class and highway fuel economy.
# geom_jitter() offsets plotted points; width = 0.2 controls horizontal jitter.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_jitter(width = 0.2)
```

14. Jittered points: example

Business use: reveal overlapping transactions or customer ratings.

استخدام تجاري: إظهار المعاملات أو تقييمات العملاء التي تتداخل نقاطها على الرسم.



Same example data and geometry as the Quarto lesson.

15. Bubble plot

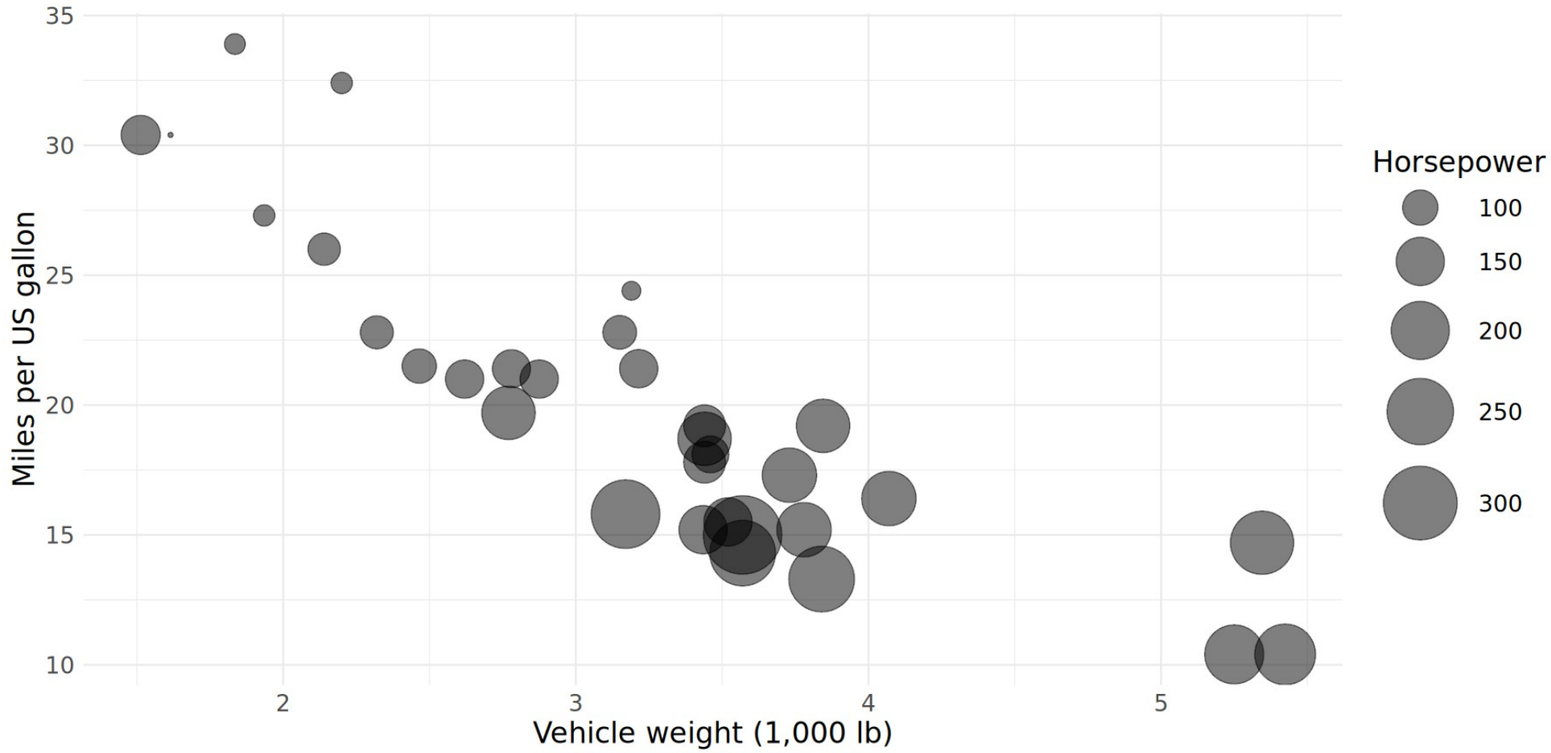
Point position encodes two variables. Point size encodes horsepower.
Transparency helps show overlap.

```
# ggplot()/aes() map variables; geom_point(alpha = 0.5) adds translucent points.
# scale_size_continuous()/c() set size range; labs() labels it; theme_minimal() styles the
# plot.
ggplot(mtcars, aes(x = wt, y = mpg, size = hp)) +
  geom_point(alpha = 0.5) +
  scale_size_continuous(range = c(1, 20)) +
  labs(size = "Horsepower") +
  theme_minimal()
```

15. Bubble plot: example

Business use: advertising and sales with transaction counts, or unit metrics with size.

استخدام تجاري: الإعلان والمبيعات مع عدد المعاملات، أو مؤشرات وحدات العمل مع الحجم.



Same example data and geometry as the Quarto lesson.

From code to a readable graphic

Choose a geometry that matches the question.
Explain mappings and units. Check transformations and defaults.
Use labels and a theme to make the evidence readable.

Continue with the ggplot2 exercises in Quarto.